CS4268 Lecture Notes

AY2025/26 Semester 2

CQT and SOC, NUS

Contents

1 Introduction

Quantum computing lies at the intersection of computer science, mathematics, and physics. It utilizes quantum mechanical laws to solve some complex problems (potentially) faster than classical computers. In this course, we shall take the laws of quantum mechanics for granted as our axiomatic "ground truth" and shall focus entirely on their **computational consequences** [NC10].

1.1 Common Computational Tasks

Let's first look at some common computational tasks and their complexities on classical computers.

Multiplication The classical primary-school (grid) algorithm for multiplying two n-digit numbers performs all digit-wise products explicitly and therefore requires a total of $O(n^2)$ single-digit operations. This was improved by Karatsuba, who introduced a divide-and-conquer strategy that reduces the multiplication of two n-digit numbers to three multiplications of size $n/2$ (with some lower-order additions). Hence we get the recursion

$$T(n) = 3T(n/2) + O(n).$$

yielding an overall time complexity of $O\big(n^{\log_2 3}\big) \approx O\big(n^{1.58}\big)$.

The fastest known classical algorithms for multiplication go even further by utilizing the Fast Fourier Transform (FFT). For example, the Schönhage–Strassen algorithm for integer multiplication achieves a runtime of $O(n \log n \log\log n)$ using FFT-based convolutions.

Factorization The simplest (and most direct) algorithm for factorizing an integer is *trial division*. It finds a nontrivial divisor by checking integers $d = 2, 3, 4, \ldots$ and stopping when $d^2 > N$. This bound works because if $N = ab$ with $a \leq b$, then $a \leq \sqrt{N}$, and if no divisor exists up to $\sqrt{N}$, then N is prime.

In the worst case, this algorithm requires $O(\sqrt{N})$ divisions, or a net $O(\sqrt{N})$ number of arithmetic operations. If the input size is n (where $n = \log_2 N$= number of bits), then

$$\sqrt{N} = 2^{\frac{1}{2}\log_2 N} = 2^{n/2}.$$

Thus, trial division has a runtime of $O(2^{n/2})$ and is exponential in the bit length of the input. Hence, this algorithm quickly becomes infeasible for practical use as the input size increases.

The most efficient known classical algorithm for factorization is the *General Number Field Sieve* (GNFS). Its runtime is

$$\exp\left(\left(\frac{64}{9}\right)^{1/3} (\log N)^{1/3} (\log\log N)^{2/3}\right),$$

which is sub-exponential in N but still super-polynomial in the input size, making it once again infeasible for practical use when N is large.

RSA
The apparent hardness of factorizing large numbers, especially semiprimes (numbers that are a product of two large primes) is what is utilized by the RSA public key cryptosystems, widely deployed in practice. It exploits the fact that given a huge semiprime N (where $N = pq$ and both p and q are large prime numbers satisfying some sanity checks), it is hard to factorize the number and find p and q.

Unstructured Database Search In the unstructured database search problem, the idea is that given a bit string $a_1a_2...a_n$, suppose it is promised that exactly one position contains 1 and rest all are 0s. The goal is to find the unique position i for which $a_i = 1$.

The classical deterministic approach to this problem is the brute force search algorithm. It checks all n possibilities in the input and finds the correct i for which $a_i = 1$ in an overall runtime of $O(n)$. In the deterministic setting, it is easy to see that $\Omega(n)$ is a lower bound as well. Even if we allow classical randomized computation, it can be shown that we still have a complexity of $\Theta(n)$ for this problem.

1.2 Probabilistic Computing

Probabilistic computing consists of randomized algorithms that, in addition to the input, also have access to a source of randomness such as independent and unbiased coin tosses. Due to the source of randomness the algorithm may lead to different execution paths for the same inputs.

Randomized algorithms can usually be divided into two types: Monte Carlo algorithms and Las Vegas algorithms. Monte Carlo algorithms are those that always terminate within a prescribed polynomial time bound but may return an incorrect answer with some bounded probability. Las Vegas algorithms on the other hand always produce a correct output, but their running time is a random variable whose expected value is usually polynomial.

Primality Testing One of the primary problems where randomized algorithms prove to be useful is primality testing. The goal here is to identify whether the input number is prime or composite. As discussed earlier, deterministic trial division takes $O(2^{n/2})$ time for this.

The Miller-Rabin test, on the other hand, uses Monte Carlo primality testing to achieve a complexity of $O(n^2 \log n \log\log n)$, which is polynomial in the input size. Solovay–Strassen [SS77] is another randomized algorithm that achieves this task in a runtime of $O(k \log^3 n)$, where k is the number of tested bases. It was only very recently (2002) that a deterministic algorithm for primality testing was found. The AKS algorithm [AKS04] provably runs in an overall time of $O(\log^{12} N)$. Subsequent improvements under specific assumptions reduces this to $O(\log^6 N)$ [LP19].

BPP = P
A very strongly held belief in the theoretical computation community is that $\mathbf{BPP} = \mathbf{P}$. This means that any problem that can be solved in polynomial time with bounded error probability (using randomness) can also be solved in polynomial time deterministically. If true, this would formalize the idea that random bits are merely a convenience for designing faster algorithms, and not a fundamentally stronger computational resource. This remains officially unproven, however.

1.3 Role of Quantum Computing

As discussed earlier, quantum computing utilizes the principles of quantum mechanics to implement algorithms that can solve specific complex problems better than classical computers. Two examples for this are as follows which we shall see in details in this course:

1. **Shor's Algorithm (1994):** Peter Shor discovered a quantum algorithm that can factor integers in **polynomial time**. This gives an exponential speedup than the best known classical counterpart.
    - *Implication:* If we can build a large-scale quantum computer, there is an "easy" way of factorizing the product of 2 large prime numbers effectively breaking the public-key RSA cryptography.
2. **Grover's Algorithm (1996):** Lov Grover discovered a way to solve the unstructured database search problem on a quantum computer in time $O(\sqrt{n})$, which can be proved to be the lower bound as well making its quantum complexity to be $\Theta(\sqrt{n})$, which is a quadratic speedup than its classical counterpart.
    - *Implication:* Its application also extends to a number of other problems, such as the Boolean Satisfiability (SAT) problem. The goal of this problem is to determine whether a Boolean formula with n variables has an assignment that makes it evaluate to true. A deterministic exhaustive search would require testing all 2^n possible assignments, whereas Grover's algorithm can find a satisfying assignment in approximately $2^{n/2}$ steps. Thus, although the runtime remains exponential, the algorithm achieves a quadratic reduction. Consequently, Grover's algorithm does not reduce NP-complete problems to polynomial time, but it effectively halves the exponent in their time complexity.

2 The Fundamental Axioms of Quantum Mechanics

In the previous section, we saw that probabilistic computing is "Classical Computing + Randomness." Quantum computing introduces something richer: interference via superposition and entanglement. A high-level summary is:

$$\text{Quantum Computing} = \text{Classical Computing} + \text{Interference (Rotation)}.$$

While probabilistic algorithms introduce randomness via coin tosses, quantum computing uses this extra structure to manipulate information in ways classical computers cannot. This is the source of quantum speedups:

- **Polynomial Speedups:** For problems like unstructured search (Grover's Algorithm).
- **Exponential Speedups:** For problems like factoring (Shor's Algorithm). [NC10]

Since quantum states behave like **vectors** that can be rotated, to understand these algorithms, we must rely heavily on **Linear Algebra**.

2.1 From Bits to Qubits

A classical bit has a state $x \in \{0, 1\}$. A quantum bit, or **qubit**, also has two basis states, which we label using the standard "ket" notation:

$$|0\rangle \quad \text{and} \quad |1\rangle .$$

For now, treat $|0\rangle$ and $|1\rangle$ as abstract labels. However, intuition comes from physics, where objects can be in more than one state at the same time:

- An **electron** may have two energy levels, labeled 0 (ground) and 1 (excited).
- A **photon** may have polarization that is Horizontal or Vertical.

If we label Horizontal as $|0\rangle$ and Vertical as $|1\rangle$, then before measurement, the photon may be in a superposition of both. After measurement, we observe only one outcome.

2.2 Law 1: Superposition

The first fundamental difference is that a qubit can be in a state that is a linear combination of the basis states.

Definition 2.1 (Quantum Mechanics Law 1: Superposition) If a quantum particle can be in one of the 2 basis states $|0\rangle$ and $|1\rangle$, then it can also be in a superposition state:

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle\,,$$

where $\alpha, \beta \in \mathbb{C}$ are complex numbers called **amplitudes**. These amplitudes must satisfy the **normalization condition**:

$$|\alpha|^2 + |\beta|^2 = 1.$$

Geometric Intuition (The Unit Circle) If we restrict ourselves to **real** amplitudes ($\alpha, \beta \in \mathbb{R}$), the condition $\alpha^2 + \beta^2 = 1$ describes a circle of radius 1. Thus, we can visualize the state of a qubit as a point on the unit circle in $\mathbb{R}^2$.

Example 2.1 Consider the state $|\psi\rangle = 0.8\,|0\rangle - 0.6\,|1\rangle$. Check normalization:

$$|0.8|^2 + |-0.6|^2 = 0.64 + 0.36 = 1.$$

Geometrically, this corresponds to the point $(0.8, -0.6)$ on the unit circle.

Remark 2.2 (Complex Amplitudes) In the general case, α and β are complex numbers ($a + ib$). The magnitude is defined as $|\alpha| = \sqrt{a^2 + b^2}$. For example, if $|\psi\rangle = \frac{1}{\sqrt{2}}\,|0\rangle + \frac{i}{\sqrt{2}}\,|1\rangle$, then $|\frac{i}{\sqrt{2}}|^2 = \frac{1}{2}$.

2.3 Law 2: Measurement

We cannot "see" the amplitudes α and β directly. We can only extract information via measurement.

Definition 2.2 (Quantum Mechanics Law 2: Measurement) Given a state $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$, if we measure it in the standard basis:

1. We observe outcome $|0\rangle$ with probability $P(0) = |\alpha|^2$.
2. We observe outcome $|1\rangle$ with probability $P(1) = |\beta|^2$.

Crucially, after the measurement, the state **collapses** to the observed outcome.

Example 2.3 Suppose we have the state $|\psi\rangle = 0.8\,|0\rangle + 0.6\,|1\rangle$.

- **Probability:** We see $|0\rangle$ with probability 0.64, and $|1\rangle$ with probability 0.36.
- **Collapse:** If we see $|0\rangle$, the state *becomes* $|0\rangle$. A second measurement will yield $|0\rangle$ with 100% probability.

This is the key distinction from simply "revealing" a hidden coin toss. In the quantum world, the measurement forces the system to choose a reality.

2.4 The Mathematical Formalism: Qubits as Vectors

So far, we have used $|0\rangle$ and $|1\rangle$ as abstract labels. Mathematically, a quantum state is a vector in a complex vector space $\mathbb{C}^d$. For a single qubit, the dimension is $d = 2$. We identify the basis states with the standard basis vectors:

$$|0\rangle = e_0 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad |1\rangle = e_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Consequently, a general superposition $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$ is simply a column vector:

$$|\psi\rangle = \alpha \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \beta \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} \alpha \\ \beta \end{bmatrix} \in \mathbb{C}^2.$$

2.5 Beyond Two-Basis States: Qudits and General Laws

There is nothing special about having exactly two states.

- A **qutrit** ($d = 3$) has basis states $|0\rangle, |1\rangle, |2\rangle$.
- A general **qudit** (d) has basis states $|0\rangle, |1\rangle, \ldots, |d-1\rangle$.

In this general setting, the state is a vector in $\mathbb{C}^d$:

$$|\psi\rangle = \alpha_0\,|0\rangle + \cdots + \alpha_{d-1}\,|d-1\rangle \iff \begin{bmatrix} \alpha_0 \\ \vdots \\ \alpha_{d-1} \end{bmatrix} \in \mathbb{C}^d.$$

Each basis state $|i\rangle$ is the standard basis vector e_i (a column with a 1 in the i-th position and 0 elsewhere).

We can now restate our two fundamental laws in their full generality.

Definition 2.3 (Law 1 in dimension d: State) A general d-dimensional state is a superposition:

$$|\psi\rangle = \sum_{i=0}^{d-1} \alpha_i\,|i\rangle, \quad \text{subject to} \quad \sum_{i=0}^{d-1} |\alpha_i|^2 = 1.$$

Definition 2.4 (Law 2 in dimension d: Measurement) Measuring $|\psi\rangle = \sum \alpha_i\,|i\rangle$ outputs outcome i with probability $|\alpha_i|^2$. Immediately after measurement, the state collapses to $|i\rangle$.

An example: a 4-dimensional system ($d = 4$) A common example is $d = 4$. We can view this as a single system with 4 levels ($|0\rangle, |1\rangle, |2\rangle, |3\rangle$), or deeply importantly, as a system of **two qubits**. If we have two qubits, the combined system has $2 \times 2 = 4$ basis states:

$$|00\rangle, |01\rangle, |10\rangle, |11\rangle.$$

Here, $|01\rangle$ means "First qubit is 0, Second qubit is 1." This scaling (2^n dimension for n qubits) is the source of quantum computing's massive state space.

3 Hilbert Space and Quantum States

The language of quantum mechanics is linear algebra. We shall assume some familiarity with it, but introduce a very useful bookkeeping tool – Dirac's bra-ket notation.

3.1 Hilbert Space

Recall the notion of a Hilbert Space.

Definition 3.1 (Hilbert Space) A **Hilbert Space** $\mathcal{H}$ over $\mathbb{C}$ is a complete vector space over the complex numbers equipped with an inner product $\langle \cdot, \cdot \rangle : \mathcal{H} \times \mathcal{H} \to \mathbb{C}$ that satisfies:

- Conjugate symmetry: $\langle u, v \rangle = \overline{\langle v, u \rangle}$
- Linearity in the second argument: $\langle u, \alpha v + \beta w \rangle = \alpha \langle u, v \rangle + \beta \langle u, w \rangle$ for $\alpha, \beta \in \mathbb{C}$
- Positive definiteness: $\langle v, v \rangle \geq 0$ with equality if and only if $v = 0$

3.2 Dirac's Bra-Ket Notation

Dirac's bra-ket notation:

Definition 3.2 (Bra-ket Notation) A **ket** $|v\rangle$ denotes a column vector in the Hilbert space $\mathcal{H}$.

- Basis vectors are denoted $|0\rangle, |1\rangle, \ldots, |n-1\rangle$ for an n-dimensional space.
- A general vector can be written as a linear combination: $|\psi\rangle = \sum_i \alpha_i |i\rangle$ where $\alpha_i \in \mathbb{C}$.
- A **bra** $\langle v|$ denotes the conjugate transpose of $|v\rangle$, i.e., $\langle v| = (|v\rangle)^\dagger$, which is a row vector.
- The inner product of two vectors $|u\rangle$ and $|v\rangle$ is written as $\langle u|v\rangle$.
- Orthonormality of basis vectors: $\langle i|j\rangle = \delta_{ij}$.

Remark 3.1 (Conjugate Transpose as Bra) Given a ket

$$|v\rangle = \begin{pmatrix} v_1 \\ \vdots \\ v_d \end{pmatrix} \in \mathbb{C}^d,$$

its bra is the conjugate transpose

$$\langle v| = (|v\rangle)^\dagger = \left(v_1^*, \ldots, v_d^*\right).$$

The inner product is therefore

$$\langle u|v\rangle = u^\dagger v,$$

which suppresses explicit matrix multiplication and should be read as "bra-u ket-v".

Example 3.2 (Notations for Bases) Dirac's notation unifies familiar notions of basis vectors:

- **High school basis (in $\mathbb{R}^3$):**

$$\hat{i} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \hat{j} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \quad \hat{k} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

- **Linear algebra basis in $\mathbb{R}^d$ (standard basis):**

$$e_1 = \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \quad e_2 = \begin{pmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{pmatrix}, \quad \ldots, \quad e_d = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{pmatrix}$$

- **Quantum computational basis in $\mathbb{C}^d$:**

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \quad |2\rangle = \begin{pmatrix} 0 \\ 0 \\ 1 \\ \vdots \\ 0 \end{pmatrix}, \quad \ldots, \quad |d-1\rangle = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{pmatrix}$$

In an n-dimensional quantum system (e.g., qubit: $n = 2$, qutrit: $n = 3$, qudit: $n = d$), these vectors form the *computational basis* $\{|0\rangle, |1\rangle, \ldots, |n-1\rangle\}$ and satisfy

$$\langle i|j\rangle = \delta_{ij}.$$

Example 3.3 (Complex Vector Example) Consider the vector

$$|v\rangle = \frac{1}{13}\begin{pmatrix} 3+4i \\ 12i \end{pmatrix}.$$

Then the corresponding bra is

$$\langle v| = \frac{1}{13}\big(3-4i,\ -12i\big),$$

obtained by taking the conjugate transpose of the ket.

3.3 Quantum States

From now on everything is denoted in bra-ket notation.

In quantum mechanics a system is mathematically modelled by a suitable (to be determined from theory and experiment) Hilbert space. The quantum state representing the system is simply an element of this Hilbert space.

Definition 3.3 (Quantum State) A **quantum state** $|\psi\rangle$ is an element of a Hilbert space $\mathcal{H}$ with unit norm, i.e., $\langle\psi|\psi\rangle = 1$.

For a quantum system with orthonormal basis $\{|0\rangle, |1\rangle, \ldots, |n-1\rangle\}$, any quantum state can be written as a superposition:

$$|\psi\rangle = \sum_{i=0}^{n-1} \alpha_i |i\rangle = \alpha_0 |0\rangle + \alpha_1 |1\rangle + \cdots + \alpha_{n-1} |n-1\rangle$$

where $\alpha_i \in \mathbb{C}$ are complex amplitudes satisfying the normalization condition $\sum_{i=0}^{n-1} |\alpha_i|^2 = 1$.

In column vector representation:

$$|\psi\rangle = \begin{pmatrix} \alpha_0 \\ \alpha_1 \\ \vdots \\ \alpha_{n-1} \end{pmatrix}.$$

Physical Interpretation: When we measure the quantum state $|\psi\rangle$ in the computational basis, the probability of observing outcome i is $|\alpha_i|^2$. This is **Born's rule**.

Example 3.4 (Quantum States as Superpositions) For a single qubit (2-dimensional quantum system):

- Computational basis states: $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$
- Superposition state: $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$
- Another superposition: $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$
- General qubit state: $|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$ where $|\alpha|^2 + |\beta|^2 = 1$

The states $|+\rangle$ and $|-\rangle$ form the Hadamard basis (also called the X-basis or plus-minus basis).

Of course, we know the basis of a (nontrivial) Hilbert space is not unique. We could choose to represent states w.r.t. different bases, so the corresponding column vectors would have different physical interpretations.

4 Measurements in different bases and the Elitzur-Vaidman Test

4.1 Measurements in Different Bases

Up until now, we have only considered measurements with respect to the standard basis ($\{|0\rangle, |1\rangle\}$ in the 2 dimensional case). However, we are not restricted to just measuring in the standard basis and can use any valid set of orthonormal basis states as the measurement basis. Let us now consider the problem of making measurements with respect to different bases.

Example 4.1 (Measurement with respect to Different Bases) Consider a qubit in state $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$.

Measurement w.r.t. computational basis $\{|0\rangle, |1\rangle\}$:

- $\Pr(\text{measure } |0\rangle) = |\frac{1}{\sqrt{2}}|^2 = \frac{1}{2}$,
- $\Pr(\text{measure } |1\rangle) = |\frac{1}{\sqrt{2}}|^2 = \frac{1}{2}$.

Measurement w.r.t. Hadamard basis $\{|+\rangle, |-\rangle\}$:

First, express $|+\rangle$ in the new basis. Since $|0\rangle = \frac{1}{\sqrt{2}}(|+\rangle + |-\rangle)$ and $|1\rangle = \frac{1}{\sqrt{2}}(|+\rangle - |-\rangle)$, we have:

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = \frac{1}{\sqrt{2}}\left[\frac{1}{\sqrt{2}}(|+\rangle + |-\rangle) + \frac{1}{\sqrt{2}}(|+\rangle - |-\rangle)\right] = 1 \cdot |+\rangle + 0 \cdot |-\rangle .$$

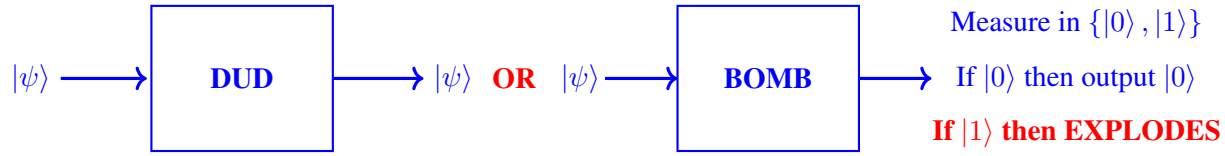
Thus

- $\Pr(\text{measure } |+\rangle) = |1|^2 = 1$,
- $\Pr(\text{measure } |-\rangle) = |0|^2 = 0$.

Key insight: When we measure $|+\rangle$ in the Hadamard basis $\{|+\rangle, |-\rangle\}$, the coefficient of $|+\rangle$ is 1 and the coefficient of $|-\rangle$ is 0. This makes intuitive sense: if the system is already in state $|+\rangle$ and we measure in the $\{|+\rangle, |-\rangle\}$ basis, we are guaranteed to get the outcome $|+\rangle$. Thus, the same quantum state gives different measurement outcomes depending on the measurement basis.

4.2 Elitzur-Vaidman Bomb-Tester

Now let us consider the following thought experiment. Suppose we are given a box that is either a DUD or a bomb. We want to figure out which box we are given without triggering an explosion. The behaviour of the two possible boxes is shown below:



Stay alive and figure out which box we are given.

Some preliminary options that comes to mind are sending $|0\rangle$ or $|+\rangle$ through the unknown box, and then measure the output from the box. Let us examine what happens in each of these cases.

Classical Strategy:

If we send $|0\rangle$, we are unable to identify if it is a dud or a bomb as the output is the same. If we send $|1\rangle$:

- No explosion $\Rightarrow$ it's a dud!
- Bomb explodes $\Rightarrow$ it's a bomb! (but you dead)

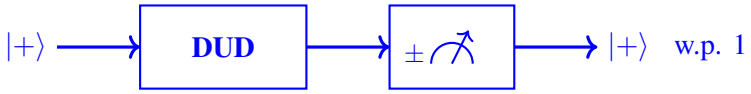
Conclusion: Classically impossible to distinguish dud/bomb without triggering an explosion.

Preliminary Quantum Strategy:

Send in $|\psi\rangle = |+\rangle$ and measure w.r.t. $\{|+\rangle, |-\rangle\}$.

Case 1: Dud bomb

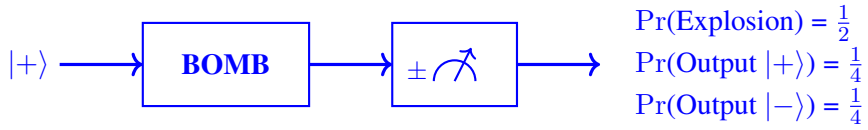
If it's a dud, we will always see $|+\rangle$ because 100% of the amplitude is on $|+\rangle$.



Case 2: Working bomb

Three possible outcomes:

- Bomb measures $|0\rangle$ and explodes (with probability $\frac{1}{2}$).
- Bomb measures $|1\rangle$ and does not explode:
  - We observe $|+\rangle$ (with probability $\frac{1}{4}$): inconclusive (could be dud or working)
  - We observe $|-\rangle$ (with probability $\frac{1}{4}$): Working bomb identified!



If we observe $|-\rangle$, we know it is a bomb because: with a dud, we get $|+\rangle$ all the time (with probability 1).

Conclusion: $\frac{1}{4}$ success rate for identifying working bomb without explosion.

Can we improve upon this? Yes. First, but first let us make a brief detour into the notion of rotations (more generally, unitaries later on) on quantum states.

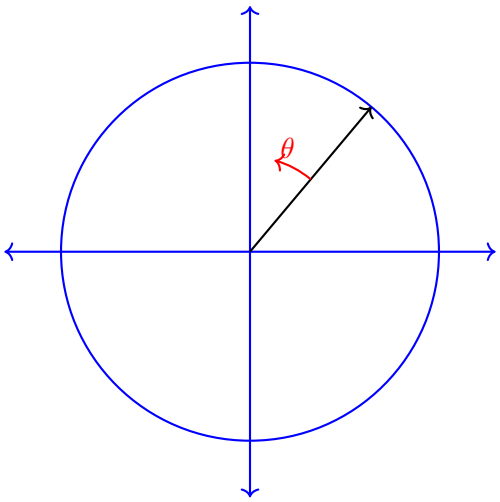
4.3 Rotations and Reflections

Rotations

Definition 4.1 (Rotations on a Single Qubit) For any θ, we can build a physical device that rotates a qubit by angle θ. Consider a transformation that rotates the quantum state by a small angle θ. Starting from $|0\rangle$, we apply a rotation: $R(\theta) : |0\rangle \mapsto \cos(\theta)\,|0\rangle + \sin(\theta)\,|1\rangle$

$$|0\rangle \to \begin{pmatrix} \cos\theta \\ \sin\theta \end{pmatrix}$$

$$|1\rangle \to \begin{pmatrix} \cos\left(\frac{\pi}{2}+\theta\right) \\ \sin\left(\frac{\pi}{2}+\theta\right) \end{pmatrix} = \begin{pmatrix} -\sin\theta \\ \cos\theta \end{pmatrix}$$



For a general state $\alpha |0\rangle + \beta |1\rangle$, it gets transformed to:

$$\begin{aligned} \alpha |0\rangle + \beta |1\rangle \to \alpha \begin{pmatrix} \cos\theta \\ \sin\theta \end{pmatrix} + \beta \begin{pmatrix} -\sin\theta \\ \cos\theta \end{pmatrix} \\ = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} \end{aligned}$$

where $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$ goes to the vector $\begin{pmatrix} \cos\theta \\ \sin\theta \end{pmatrix}$ and $\begin{pmatrix} 0 \\ 1 \end{pmatrix}$ goes to the vector $\begin{pmatrix} -\sin\theta \\ \cos\theta \end{pmatrix}$.

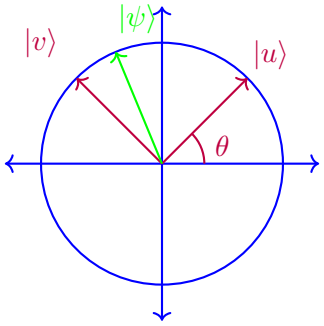
Any state $|\psi\rangle$ gets transformed to $R_\theta |\psi\rangle$, where:

$$R_\theta = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

Note:

- α and β may be complex numbers.
- Viewing R_θ as a rotation may only be intuitive for real amplitudes.

Measuring w.r.t. $|u\rangle, |v\rangle$ basis via rotation:



$$|u\rangle = R_\theta |0\rangle$$
$$|v\rangle = R_\theta |1\rangle$$

For any state:

$$\begin{aligned} |\psi\rangle &= \alpha |u\rangle + \beta |v\rangle \\ &= \alpha \cdot R_\theta |0\rangle + \beta \cdot R_\theta |1\rangle \end{aligned}$$

How to measure in $|u\rangle, |v\rangle$ while actually measuring in standard basis?

Apply $R_{-\theta}$ to $|\psi\rangle$:

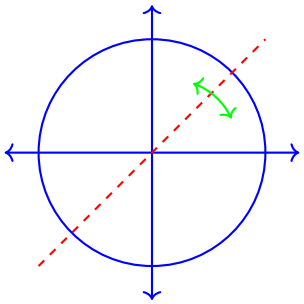
$$R_{-\theta} |\psi\rangle = \alpha |0\rangle + \beta |1\rangle$$

Then measure in $\{|0\rangle, |1\rangle\}$:

- $|0\rangle$ with probability $|\alpha|^2 \rightarrow$ output $|u\rangle$
- $|1\rangle$ with probability $|\beta|^2 \rightarrow$ output $|v\rangle$.

Reflections We can also build reflectors along any axis.

Example 1: Reflection along diagonal axis



Reflection transformations:

$$\begin{pmatrix} 1 \\ 0 \end{pmatrix} \rightarrow \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad \begin{pmatrix} 0 \\ 1 \end{pmatrix} \rightarrow \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$
$$|0\rangle \rightarrow |1\rangle, \quad |1\rangle \rightarrow |0\rangle$$

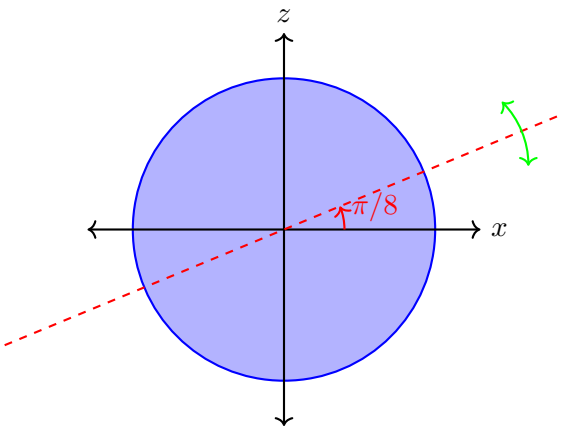
For a general state:

$$\begin{pmatrix} \alpha \\ \beta \end{pmatrix} \text{ gets transformed to } \begin{pmatrix} \beta \\ \alpha \end{pmatrix}$$

NOT operation:

$$|\psi\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix} \text{ maps to } \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix}$$

Example 2: Hadamard gate (reflection along different axis)



Hadamard transformations:

$$|0\rangle \text{ maps to } |+\rangle$$
$$|1\rangle \text{ maps to } |-\rangle$$

$$\psi = \begin{pmatrix} \alpha \\ \beta \end{pmatrix} \text{ is mapped to } \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix}$$

This is labeled the Hadamard gate and is arguably the most important transformation in quantum computing.

4.4 Elitzur-Vaidman Revisited

Improved Quantum Strategy via small-angle rotations (Quantum Zeno effect) We now show how to make the explosion probability arbitrarily small. Let $\varepsilon = \frac{\pi}{2N} > 0$ be a small angle, i.e. N is large. Define the rotation operator

$$R_\varepsilon = \begin{pmatrix} \cos\varepsilon & -\sin\varepsilon \\ \sin\varepsilon & \cos\varepsilon \end{pmatrix}.$$

Starting from $|0\rangle$, we perform the following protocol:

1. Apply R_ε.
2. Send the qubit through the unknown box.
3. If no explosion occurs, repeat the above steps N times.
4. Finally, measure in the computational basis.

Dud: Since $N = \frac{\pi}{2\varepsilon}$, in the dud case the cumulative rotation is

$$R_\varepsilon^N = R_{\pi/2},$$

which maps $|0\rangle$ close to $|1\rangle$. Thus after N rounds the state is approximately $|1\rangle$, and a final measurement yields outcome $|1\rangle$ with full certainty.

Working Bomb: After a single application of R_ε the state becomes $\cos\varepsilon\,|0\rangle + \sin\varepsilon\,|1\rangle$. The box then measures in the computational basis. The probability of explosion in that round is $\sin^2\varepsilon \le \varepsilon^2$. If no explosion occurs, the state collapses back to $|0\rangle$, and the process repeats.

By the union bound, the probability of at least one explosion over N rounds is upper-bounded by $N\varepsilon^2 = \frac{\pi}{2}\varepsilon$, tending to 0 as $\varepsilon \to 0$. Meanwhile, if no explosion occurs for any of the N rounds, the final measurement yields $|0\rangle$, in stark contrast to the dud case, which yields $|1\rangle$. Thus by choosing ε sufficiently small, we can distinguish a live bomb from a dud with arbitrarily high confidence, while making the probability of explosion arbitrarily small.

In a nutshell, using this quantum Zeno strategy we have

$$\Pr(\text{observe } |1\rangle \mid \text{DUD }) = 1$$
$$\Pr(\text{observe } |0\rangle \mid \text{BOMB }) \geq 1 - \frac{\pi}{2}\varepsilon.$$

5 Unitary Operations

We have already seen some operations like the **NOT** operation and the **Hadamard** operation. We will now understand what kind of transformations are allowed on quantum states. The transformations that are physically allowed on quantum states are called unitary transformations (or operations). Intuitively, unitaries preserve the 'length' of quantum states, thus preserving probability densities.

Quantum Mechanics Law 3: Quantum states can be transformed (without losing information) via **unitary transformations**.

Definition 5.1 (Unitary Matrices) A linear transformation $U : \mathcal{H} \to \mathcal{H}$ on a complex Hilbert space is called **unitary** if it preserves the norm of every state. That is,

$$\|U|\psi\rangle\|^2 = \||\psi\rangle\|^2 \quad \text{for all } |\psi\rangle \in \mathcal{H}.$$

Here are a few examples of unitary matrices:

Example 5.1 (Examples of Unitary Matrices)

1. **Identity:** $I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$
2. **Pauli-X (NOT gate):** $X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$

   Effect: $X|0\rangle = |1\rangle$ and $X|1\rangle = |0\rangle$ (bit flip)
3. **Pauli-Y:** $Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$
4. **Pauli-Z (phase flip):** $Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$

   Effect: $Z|0\rangle = |0\rangle$ and $Z|1\rangle = -|1\rangle$
5. **Hadamard gate:** $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$

Effect: Creates equal superpositions

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle$$
$$H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle$$

Note: $H^2 = I$ (Hadamard is self-inverse).

6. **Phase gate (S gate):** $S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$
7. **T gate ($\pi/8$ gate):** $T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$
8. **Rotation gates:** $R_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}$

5.1 Properties of Unitaries

Let us present the various characterizations of unitary matrices.

Proposition 5.1 (Basic Properties of Unitary Matrices)

1. **Preservation of inner products and norms)**: for any $|v\rangle, |w\rangle$: $\langle Uv|Uw\rangle = \langle v|U^\dagger U|w\rangle = \langle v|w\rangle$. In particular, $\|U|v\rangle\|^2 = \||v\rangle\|^2$.
2. **Preservation of angles**: Since $\cos(\theta) := \frac{\langle v|w\rangle}{\|v\|\|w\|}$ and unitary transformations preserve both inner products and norms, the angle between any two vectors is also preserved.
3. **Eigenvalues lie on unit circle:** If $U|v\rangle = \lambda|v\rangle$, then $|\lambda| = 1$. *Proof:* $\||v\rangle\|^2 = \|U|v\rangle\|^2 = \|\lambda|v\rangle\|^2 = |\lambda|^2\||v\rangle\|^2$, so $|\lambda| = 1$.
4. **Determinant has unit modulus:** $|\det(U)| = 1$. *Proof:* take determinants on both sides of $U^\dagger U = I$.
5. **Composition of unitaries is unitary:** If U, V are unitary, then $(UV)^\dagger(UV) = V^\dagger U^\dagger UV = V^\dagger V = I$.
6. **Inverse is unitary:** If U is unitary, so is $U^\dagger = U^{-1}$.
7. **Reversibility:** Quantum evolution is reversible. Given $|\psi'\rangle = U|\psi\rangle$, we can always recover $|\psi\rangle = U^{-1}|\psi'\rangle = U^\dagger|\psi'\rangle$.
8. **Square roots exist:** Every unitary U has a square root W, which is also unitary, such that $W^2 = U$. This allows for fractional quantum operations.

   Example: Consider the NOT gate (Pauli-X): $X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$. Its square root is:

$$\sqrt{X} = \frac{1}{2}\begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix}$$

We can verify: $\sqrt{X} \cdot \sqrt{X} = \frac{1}{4}\begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix}\begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = X$

On the Bloch sphere, $\sqrt{X}$ represents a rotation halfway to the full NOT operation.

5.2 Classical Analogue of Unitary Transformations

It is instructive to compare unitary evolution in quantum mechanics with the way probability distributions evolve in classical randomized processes. In the classical setting, a system with d possible outcomes can be represented by a probability vector

$$p = \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_d \end{bmatrix}, \qquad p_i \geq 0, \qquad \sum_{i=1}^{d} p_i = 1.$$

A randomized procedure transforms p into a new distribution p' via multiplication by a matrix M:

$$p' = Mp.$$

To preserve total probability, such a matrix must be column-stochastic, meaning its entries are non-negative and each column sums to 1:

$$M_{ij} \geq 0 \qquad \text{and} \qquad \sum_{i=1}^{d} M_{ij} = 1 \text{ for all } j.$$

We now illustrate this with a classical example.

Consider the following random process: we roll a fair die; if the outcome is odd, we toss a fair coin; if it's heads, add 1 to the outcome of the fair die.

Let the state space be $\{1, 2, 3, 4, 5, 6\}$, and start from the uniform distribution

$$p = \begin{bmatrix} \frac{1}{6} \\ \frac{1}{6} \\ \frac{1}{6} \\ \frac{1}{6} \\ \frac{1}{6} \\ \frac{1}{6} \end{bmatrix}.$$

The transformation implemented by the above procedure is captured by the column-stochastic matrix

$$M = \begin{bmatrix} \frac{1}{2} & 0 & 0 & 0 & 0 & 0 \\ \frac{1}{2} & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{2} & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{2} & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{2} & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{2} & 1 \end{bmatrix}, \qquad p' = Mp.$$

Starting from an odd face $1, 3, 5$, the coin toss either keeps the value (tails) or moves it to the next even face (heads), giving the $\frac{1}{2}$–$\frac{1}{2}$ split in the corresponding column. Starting from an even face $2, 4, 6$, the value stays unchanged, which is why the corresponding column has a single 1.

This classical picture is quite similar to quantum evolution in that both are linear transformations on vectors. However, there are key differences.

First, classical stochastic evolution preserves the ℓ_1 norm,

$$\|p'\|_1 = \|p\|_1 = 1,$$

whereas quantum evolution preserves the ℓ_2 norm,

$$\|U\,|\psi\rangle\,\|_2 = \|\,|\psi\rangle\,\|_2.$$

Second, classical stochastic transformations are generally not reversible: a column-stochastic matrix need not be invertible, and even when it is invertible, its inverse typically does not have non-negative entries and therefore does not correspond to a valid stochastic process. In contrast, every unitary matrix is reversible, with inverse $U^\dagger$ as seen in the characterisation. This reversibility is one of the fundamental distinctions between classical and quantum dynamics.

6 Composite Systems

We have discussed how to describe a single quantum system via the notion of quantum states residing in a Hilbert space, how they can be transformed and how information is extracted from the system via measurements. Now we discuss how to characterize *composite* systems. Given two (or more) quantum systems, each with its own Hilbert space, how do we describe them collectively as a whole? In particular, if we only effect changes on a component of this composite system, how can we describe this mathematically as an operation on the entire system?

This is achieved via the notion of *tensor products*.

6.1 Tensor Products

Let us consider a system comprising simply two qudits. This system is mathematically described by the tensor product of the Hilbert spaces of the individual qudits:

Definition 6.1 (Tensor Product of Hilbert Spaces) Given two qudit Hilbert spaces $\mathcal{H}_1 = \mathbb{C}^{d_1}$, $\mathcal{H}_2 = \mathbb{C}^{d_2}$ spanned respectively by the computational basis vectors

$$\mathcal{B}_1 = \{|0\rangle, |1\rangle, \ldots, |d_1 - 1\rangle\}, \quad \mathcal{B}_2 = \{|0\rangle, |1\rangle, \ldots, |d_2 - 1\rangle\},$$

the **tensor product** of $\mathcal{H}_1$ and $\mathcal{H}_2$ is defined as the Hilbert space

$$\mathcal{H}_1 \otimes \mathcal{H}_2 = \text{span}\{|i\rangle \otimes |j\rangle : 0 \leq i \leq d_1 - 1, 0 \leq j \leq d_2 - 1\}.$$

Each (normalized) state $|\psi\rangle = \sum_{ij} a_{ij} |i\rangle \otimes |j\rangle \in \mathcal{H}_1 \otimes \mathcal{H}_2$, $\sum_{ij} |a_{ij}|^2 = 1$, is a legitimate characterization of the entire system. Also, for brevity we shall henceforth write $|\alpha\rangle \otimes |\beta\rangle$ simply as $|\alpha\beta\rangle$.

More generally, for $n > 2$ systems, the tensor product generalizes naturally to

$$\mathcal{H}_1 \otimes \mathcal{H}_2 \otimes \cdots \otimes \mathcal{H}_n = \text{span}\{|i_1\rangle \otimes \cdots \otimes |i_n\rangle\}.$$

Remark 6.1 (Existence and Uniqueness of Tensor Products) Note that above we have defined the tensor product as being the span of the *formal* symbols $|i\rangle \otimes |j\rangle$ – we have not at all dissected what the symbols $|i\rangle \otimes |j\rangle$ really mean. In particular, what is the difference between $|i\rangle \otimes |j\rangle$ and the *direct sum* $(|i\rangle, |j\rangle)$? Superficially, both seem quite similar in the sense that the individual constituents are simply concatenated together (albeit in different notations). Indeed, these two notions are closely related: roughly speaking, we think of the symbol $\otimes$ as a *bilinear map* from the direct sum/Cartesian product $H_1 \times H_2$ to the Hilbert space Z (we use the symbol Z because we have not yet proved that this thing exists). That is, the purported map $\otimes$ behaves as such (on all vectors $|u\rangle, |v\rangle$, not just the basis vectors):

$$\otimes(|u\rangle, |v\rangle) := |u\rangle \otimes |v\rangle$$

such that

$$(a\,|u_1\rangle + b\,|u_2\rangle) \otimes |v\rangle = a\,|u_1\rangle \otimes |v\rangle + b\,|u_2\rangle \otimes |v\rangle \tag{6.1}$$
$$|u\rangle \otimes (a\,|v_1\rangle + b\,|v_2\rangle) = a\,|u\rangle \otimes |v_1\rangle + b\,|u\rangle \otimes |v_2\rangle\,. \tag{6.2}$$

The existence and (essential) uniqueness of the map $\otimes$ is nontrivial to show. Once these are taken care of we can start calling Z as $H_1 \otimes H_2$. If you are interested, the wiki article on tensor products is a good starting point. Be warned however that this leads to a long rabbit hole.

That was a lot of yapping – but we won't worry about any of that in this course. In practice, think of $\otimes$ as concatenation plus multilinearity: the symbol $\otimes$ is used to stack different basis vectors from different Hilbert spaces together, and that $\otimes$ is linear with respect to *all* its arguments, i.e. Eq. (6.1)

Fact 6.1 (Column Vector Representation of Tensor Products) Given vectors $|\alpha\rangle \in \mathcal{H}_1$ and $|\beta\rangle \in \mathcal{H}_2$, the column representation of $|\alpha\rangle \otimes |\beta\rangle$ is given by

$$|a\rangle \otimes |b\rangle = \begin{pmatrix} a_0 \\ \vdots \\ a_{d_1-1} \end{pmatrix} \otimes \begin{pmatrix} b_0 \\ \vdots \\ b_{d_2-1} \end{pmatrix} = \begin{pmatrix} a_0 b_0 \\ \vdots \\ a_0 b_{d_2-1} \\ \vdots \\ a_{d_1-1} b_0 \\ \vdots \\ a_{d_1-1} b_{d_2-1} \end{pmatrix}.$$

In particular, the tensor product of the basis vectors is given by

$$|i\rangle \otimes |j\rangle = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

where the 1 appears at position $i \cdot d_2 + j$.

These results are immediate from the definition of the tensor product.

Example 6.2 For two qubits ($d_1 = d_2 = 2$), the computational basis states are represented as

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

The superposition states are

$$|+\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \qquad |-\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

We now give examples of tensor products in column-vector form:

$$|0\rangle \otimes |0\rangle = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix},$$

$$|0\rangle \otimes |+\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix},$$

$$|+\rangle \otimes |0\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix},$$

$$|+\rangle \otimes |-\rangle = \frac{1}{2} \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix}.$$

More generally, suppose Alice has qubit $|A\rangle = \alpha_0 |0\rangle + \alpha_1 |1\rangle$ and Bob has qubit $|B\rangle = \beta_0 |0\rangle + \beta_1 |1\rangle$. Then their joint state is

$$|A\rangle \otimes |B\rangle = \alpha_0\beta_0 |0\rangle \otimes |0\rangle + \alpha_0\beta_1 |0\rangle \otimes |1\rangle + \alpha_1\beta_0 |1\rangle \otimes |0\rangle + \alpha_1\beta_1 |1\rangle \otimes |1\rangle .$$

Notice how the multilinearity (bilinearity here) was invoked to obtain this expansion. In column vector form:

$$|A\rangle \otimes |B\rangle = \begin{bmatrix} \alpha_0\beta_0 \\ \alpha_0\beta_1 \\ \alpha_1\beta_0 \\ \alpha_1\beta_1 \end{bmatrix}.$$

To verify that the composite coefficients are indeed probability amplitudes, we check if the sum of amplitudes squared is 1. But this is easy – according to Quantum Mechanics Law 1, the amplitudes of $|A\rangle$ and $|B\rangle$ satisfy the **normalization condition** i.e. $|\alpha_0|^2 + |\alpha_1|^2 = 1$ and $|\beta_0|^2 + |\beta_1|^2 = 1$, so

$$\begin{aligned} |\alpha_0\beta_0|^2 + |\alpha_0\beta_1|^2 + |\alpha_1\beta_0|^2 + |\alpha_1\beta_1|^2 &= |\alpha_0|^2(|\beta_0|^2 + |\beta_1|^2) + |\alpha_1|^2(|\beta_0|^2 + |\beta_1|^2) \\ &= |\alpha_0|^2 + |\alpha_1|^2 \\ &= 1 \end{aligned}$$

as desired.

Definition 6.2 (Tensor Product of Matrices) Given matrices $A \in \mathbb{C}^{m\times n}$ and $B \in \mathbb{C}^{p\times q}$, their tensor (also called Kronecker) product $A \otimes B$ is the $mp \times nq$ matrix

$$A \otimes B = \begin{bmatrix} a_{11}B & a_{12}B & \cdots & a_{1n}B \\ a_{21}B & a_{22}B & \cdots & a_{2n}B \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}B & a_{m2}B & \cdots & a_{mn}B \end{bmatrix}.$$

Remark 6.3 We seem to have two different (but superficially very similar) notions of tensor products, one for vectors and one for matrices, but note that technically the set of matrices itself is an abstract vector space, thus there really is no difference on an abstract level. The only difference here of course is the *concrete representation* of the objects. What we call vectors in this course we generally write down as columns (long-ish 1D objects), while the operators on these vectors (which technically speaking also form a vector space themselves) we write down as matrices (square-ish 2D objects). Hopefully this isn't too confusing. If it helps you can restack the rectangular-looking matrices into long columns (vectorization). Then the basis matrices spanning matrix space gets rewritten as long, all-zeros-except-one vectors. Then this should be very clear that the tensor product of matrices is exactly the same as the general definition given in Definition 6.1.

Proposition 6.2 The tensor product of matrices satisfies the following properties:

- $(A \otimes B)(C \otimes D) = (AC) \otimes (BD)$ whenever dimensions match.
- $(A \otimes B)^\dagger = A^\dagger \otimes B^\dagger$.
- $\mathrm{Tr}(A \otimes B) = \mathrm{Tr}(A)\mathrm{Tr}(B)$.
- If A and B are invertible, then $(A \otimes B)^{-1} = A^{-1} \otimes B^{-1}$.

Why are matrix tensor products useful? Recall that matrices appear a lot in quantum, as operations on the quantum states (unitaries, measurements etc). Matrix tensor products characterize the *joint* application of these operations on the entire system. Consider a two qubit basis state $|\psi\rangle = |00\rangle$. Suppose we apply a unitary transformation $U = \begin{bmatrix} p & r \\ q & s \end{bmatrix}$ only to the second qubit.

The overall transformation on $|\psi\rangle$ can be described by

$$|00\rangle \to |0\rangle \otimes (U|0\rangle)$$

The transformations for the other basis states can be obtained similarly. Specifically:

$$|00\rangle \to |0\rangle \otimes (U|0\rangle) = \begin{bmatrix} p \\ q \\ 0 \\ 0 \end{bmatrix}$$

$$|01\rangle \to |0\rangle \otimes (U|1\rangle) = \begin{bmatrix} r \\ s \\ 0 \\ 0 \end{bmatrix}$$

$$|10\rangle \to |1\rangle \otimes (U|0\rangle) = \begin{bmatrix} 0 \\ 0 \\ p \\ q \end{bmatrix}$$

$$|11\rangle \to |1\rangle \otimes (U|1\rangle) = \begin{bmatrix} 0 \\ 0 \\ r \\ s \end{bmatrix}.$$

Denoting I as the identity matrix, this transformation can be succinctly written as

$$I \otimes U \left|\psi\right\rangle$$

where

$$I \otimes U = \begin{bmatrix} p & r & 0 & 0 \\ q & s & 0 & 0 \\ 0 & 0 & p & r \\ 0 & 0 & q & s \end{bmatrix}$$

Intuitively, the 'do nothing' operation on the first qubit is mathematically represented as applying the identty operator on the first qubit.

Theorem 6.3 (Single Qubit Unitary Transformations in Composite System) Suppose U, V are unitary transformations. If we apply U to $\left|\phi\right\rangle$ and V to $\left|\psi\right\rangle$, the joint state $\left|\phi\psi\right\rangle$ is transformed to $U \otimes V \left|\phi\psi\right\rangle$.

Proof. We can consider the unitary transformations applied to $\left|\phi\right\rangle$ and $\left|\psi\right\rangle$ as two separate unitary transformations applied consecutively:

$$\begin{aligned} (I \otimes V)(U \otimes I) \left|\phi\psi\right\rangle &= (IU) \otimes (VI) \left|\phi\psi\right\rangle \quad ((A \otimes B)(C \otimes D) = (AC) \otimes (BD)) \\ &= (U \otimes V) \left|\phi\psi\right\rangle . \end{aligned}$$

□

Next we consider an extremely important gate in quantum computing, the **CNOT gate**. A CNOT gate takes in two qubits, a control qubit and a target qubit. Denote the control qubit as $\left|\phi\right\rangle$ and the target qubit as $\left|\psi\right\rangle$.

- If the control qubit is 0, do nothing.
- If the control qubit is 1, flip the target qubit.

Given $\left|\phi\psi\right\rangle$, enumerating over all the basis states, we have the following transformations effected by CNOT:

$$\begin{aligned} \left|00\right\rangle &\to \left|00\right\rangle \\ \left|01\right\rangle &\to \left|01\right\rangle \\ \left|10\right\rangle &\to \left|11\right\rangle \\ \left|11\right\rangle &\to \left|10\right\rangle . \end{aligned}$$

Note that the resulting basis is still orthonormal. The CNOT gate can be represented by the following matrix,

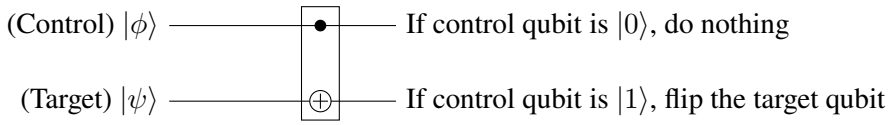
$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} .$$

The first two columns in the CNOT matrix represents the transformation to $\left|\psi\right\rangle$ when $\phi = 0$, while the last two columns in the CNOT matrix represents the "flipping" transformation to $\left|\psi\right\rangle$ when $\phi = 1$.

The probability amplitudes of the original basis will be mapped to the corresponding basis after the CNOT operation. More concretely,

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} \alpha_{00} \\ \alpha_{01} \\ \alpha_{10} \\ \alpha_{11} \end{bmatrix} = \begin{bmatrix} \alpha_{00} \\ \alpha_{01} \\ \alpha_{11} \\ \alpha_{10} \end{bmatrix}.$$

CNOT as a diagram:

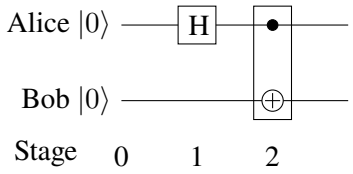


The next example on creating EPR states is intended to be a hands-on exercise to help you gain familiarity with tensor product manipulation.

Example 6.4 (Creating EPR/Bell states) To produce the EPR state $|\Phi^+\rangle := \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$, we use a circuit consisting of a Hadamard gate (H) followed by a CNOT:

1. **Initial State:** $|\psi_0\rangle = |0\rangle \otimes |0\rangle = |00\rangle$.
2. **Apply H to first qubit:** $|\psi_1\rangle = (H \otimes I)\,|00\rangle = \left(\frac{|0\rangle+|1\rangle}{\sqrt{2}}\right) \otimes |0\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)$.
3. **Apply CNOT:** $|\psi_2\rangle = \text{CNOT}\left(\frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)\right) = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$.

Diagrammatically,



6.2 Quantum Entanglement

Look at the Bell state again. We might think: what is the appropriate $|\psi\rangle$ and $|\phi\rangle$ such that $|\Phi^+\rangle = |\psi\rangle \otimes |\phi\rangle$? That is, what are the individual states of the qubits that collectively are described as the Bell state $|\Phi^+\rangle$ on the entire system?

Answer: there are no pairs of individual states $|\phi\rangle$ and $|\psi\rangle$ such that $|\Phi^+\rangle = |\phi\rangle \otimes |\psi\rangle$.

Proof. Assume that $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ is separable, i.e. there exist states $|\psi\rangle = \alpha_0\,|0\rangle + \alpha_1\,|1\rangle$ and $|\phi\rangle = \beta_0\,|0\rangle + \beta_1\,|1\rangle$ such that

$$\left|\Phi^+\right\rangle = |\psi\rangle \otimes |\phi\rangle\,.$$

Expanding the tensor product out gives

$$|\psi\rangle \otimes |\phi\rangle = (\alpha_0\,|0\rangle + \alpha_1\,|1\rangle) \otimes (\beta_0\,|0\rangle + \beta_1\,|1\rangle)$$

$$= \alpha_0\beta_0 |00\rangle + \alpha_0\beta_1 |01\rangle + \alpha_1\beta_0 |10\rangle + \alpha_1\beta_1 |11\rangle$$

We compare the coefficients of this expansion to the Bell state $\frac{1}{\sqrt{2}} |00\rangle + 0 |01\rangle + 0 |10\rangle + \frac{1}{\sqrt{2}} |11\rangle$:

1. $\alpha_0\beta_0 = \frac{1}{\sqrt{2}}$ (from $|00\rangle$)
2. $\alpha_0\beta_1 = 0$ (from $|01\rangle$)
3. $\alpha_1\beta_0 = 0$ (from $|10\rangle$)
4. $\alpha_1\beta_1 = \frac{1}{\sqrt{2}}$ (from $|11\rangle$)

From (2), either $\alpha_0 = 0$ or $\beta_1 = 0$.

- If $\alpha_0 = 0$, then $\alpha_0\beta_0 = 0$, which contradicts (1).
- If $\beta_1 = 0$, then $\alpha_1\beta_1 = 0$, which contradicts (4).

Since both possibilities lead to a contradiction, no such α_i, β_i exist. Therefore, $|\Phi^+\rangle$ is nonseparable. □

We say that $|\Phi^+\rangle$ is an **entangled** state. (Quantum) Entanglement is a *core* feature of quantum mechanics and quantum computing, but mathematically it simply expresses the non-separability of the system state.

Definition 6.3 (Separability and Entanglement) Consider a composite quantum system described by the Hilbert space $\mathcal{H} = \mathcal{H}_A \otimes \mathcal{H}_B$.

- A pure state $|\Psi\rangle \in \mathcal{H}$ is said to be **separable** (or non-entangled) if it can be written as a tensor product of states from the individual subsystems:

  $$|\Psi\rangle = |\psi\rangle_A \otimes |\phi\rangle_B$$

  where $|\psi\rangle_A \in \mathcal{H}_A$ and $|\phi\rangle_B \in \mathcal{H}_B$.
- A pure state $|\Psi\rangle$ is said to be **entangled** if it cannot be decomposed in such a manner. That is:

  $$|\Psi\rangle \neq |\psi\rangle_A \otimes |\phi\rangle_B \quad \forall |\psi\rangle_A, |\phi\rangle_B$$

While an entangled system state cannot be written as a tensor product of subsystem states, we *can* actually make sense of and talk about these subsystem states, via the concept of partial trace. More on that later in the course.

6.3 Partial Measurements

So far we have seen how to describe composite systems and the operations on them, generally speaking. Performing measurements is also a type of operation. In this section we briefly explore the notion of partial measurements, i.e. making measurements only on a subsystem of the whole system.

Intuitively, if Alice measures *only her qubit* in a joint state, she should obtain a classical outcome (here 0 or 1) according to the Born rule, and the *post-measurement joint state* should be the part of the superposition consistent with that outcome, renormalized to have norm 1.

Consider a general two-qubit state (Alice is the first qubit, Bob is the second):

$$|\psi\rangle = \alpha_{00}|00\rangle + \alpha_{01}|01\rangle + \alpha_{10}|10\rangle + \alpha_{11}|11\rangle, \qquad \sum_{i,j\in\{0,1\}} |\alpha_{ij}|^2 = 1.$$

Alice measures her qubit (in the computational basis). We have the case

$$\Pr[\text{Alice reads } |0\rangle] = |\alpha_{00}|^2 + |\alpha_{01}|^2$$

wherein the state collapses to

$$\frac{\alpha_{00}|00\rangle + \alpha_{01}|01\rangle}{\sqrt{|\alpha_{00}|^2 + |\alpha_{01}|^2}} = \gamma_0|00\rangle + \gamma_1|01\rangle,$$

where

$$\gamma_0 = \frac{\alpha_{00}}{\sqrt{|\alpha_{00}|^2 + |\alpha_{01}|^2}}, \qquad \gamma_1 = \frac{\alpha_{01}}{\sqrt{|\alpha_{00}|^2 + |\alpha_{01}|^2}};$$

and the case

$$\Pr[\text{Alice reads } |1\rangle] = |\alpha_{10}|^2 + |\alpha_{11}|^2$$

wherein the state collapses to

$$\frac{\alpha_{10}|10\rangle + \alpha_{11}|11\rangle}{\sqrt{|\alpha_{10}|^2 + |\alpha_{11}|^2}} = \gamma'_0|10\rangle + \gamma'_1|11\rangle,$$

where

$$\gamma'_0 = \frac{\alpha_{10}}{\sqrt{|\alpha_{10}|^2 + |\alpha_{11}|^2}}, \qquad \gamma'_1 = \frac{\alpha_{11}}{\sqrt{|\alpha_{10}|^2 + |\alpha_{11}|^2}}.$$

The amplitudes γ_0, γ_1 (respectively γ'_0, γ'_1) are the *conditional* probability amplitudes of the two-qubit basis states given Alice's outcome. Equivalently, they describe Bob's post-measurement state conditioned on Alice:

$$|\psi_B \mid A = 0\rangle = \gamma_0|0\rangle + \gamma_1|1\rangle, \qquad |\psi_B \mid A = 1\rangle = \gamma'_0|0\rangle + \gamma'_1|1\rangle,$$

since in the $A = 0$ branch the joint state is $|0\rangle \otimes (\gamma_0|0\rangle + \gamma_1|1\rangle)$, and similarly for $A = 1$.

Physically, Alice's measurement *projects* the joint state onto the subspace consistent with her observed classical bit and the renormalization ensures the remaining state has total probability 1. If $|\psi\rangle$ is a product state, measuring Alice does not change Bob's qubit. If $|\psi\rangle$ is entangled, then Bob's conditional state can depend on Alice's outcome, which is the source of the strong correlations observed in entangled measurements.

7 The No-Cloning Theorem and Quantum Teleportation

Every single week during the lecture, a few of you open this overleaf file and copy the scribe template over to your own file to work on it. Some of you use ChatGPT to help refine your scribes (which is okay, just be honest with it). The numerous ctrl-C's/ctrl-V's involved in these undertakings were in all likelihood barely noticed. But they of course were instances of copying/cloning (classical) information, which is ubiquitous. Unfortunately, the same cannot be done with *quantum* information.

7.1 No Quantum Clones

Theorem 7.1 There does not exist a unitary U which upon arbitrary input states $|\psi\rangle$ outputs $|\psi\rangle|\psi\rangle$. More precisely, a unitary such that

$$U|\psi\rangle|\text{anc}\rangle = |\psi\rangle|\psi\rangle \quad \forall |\psi\rangle$$

does not exist. Here $|\text{anc}\rangle$ denotes *any arbitrary* ancillary state which has the same dimension as $|\psi\rangle$.

Proof. We present two proofs.

Proof 1: Suppose there exists such a unitary U. Pick an arbitrary state $|\psi\rangle = a|0\rangle + b|1\rangle$ where a, b are complex amplitudes. Running U on $|\psi\rangle$ yields

$$U(a|0\rangle + b|1\rangle)|\text{anc}\rangle = a^2|00\rangle + ab|01\rangle + ab|10\rangle + b^2|11\rangle.$$

But at the same time we also have $U|0\rangle|\text{anc}\rangle = |00\rangle$ and $U|1\rangle|\text{anc}\rangle = |11\rangle$. By linearity we thus have

$$U(a|0\rangle + b|1\rangle)|\text{anc}\rangle = a|00\rangle + b|11\rangle.$$

Matching the two given expressions for $U|\psi\rangle|\text{anc}\rangle$ we see that they are compatible only when $(a, b) = (0, 1)$ or $(1, 0)$, contradicting the arbitrariness of $|\psi\rangle$.

Proof 2: Pick two arbitrary states $|\psi\rangle, |\phi\rangle$ and run U on them. This gives $U|\psi\rangle|\text{anc}\rangle = |\psi\rangle|\psi\rangle$ and $U|\phi\rangle|\text{anc}\rangle = |\phi\rangle|\phi\rangle$. Taking their inner products and noting the unitarity of U we see that $\langle\psi|\phi\rangle = \langle\psi|\phi\rangle^2 \implies \langle\psi|\phi\rangle = 0/1$. That is, the two states must either be the same or mutually orthogonal, once again contradicting the assumed arbitrariness. □

7.2 Quantum Teleportation

Consider the following scenario: suppose we have 2 friends, Alice and Bob, and they live far away from one another. Alice has in her possession some unknown qubit $|\psi\rangle := \alpha|0\rangle + \beta|1\rangle$ and she wants to send it to Bob. How can she do it?

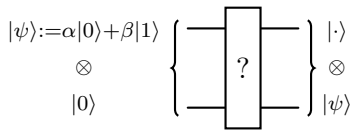


Figure 1: Some unknown quantum teleporter

A Preliminary Puzzle Before we start off showing how it can be done, let us start off with a simpler puzzle. Suppose Alice and Bob are sitting in the lab, and Alice wants to send some unknown qubit $|\psi\rangle$ to Bob. Obviously, she cannot walk up to Bob to give it to him but what they do have is a shared network for both of them to perform unitary operations (see Figure 1).

Note that from Figure 1, we want these states to be un-entangled so that Alice cannot "destroy" the qubit via a measurement.

What Alice can do is to apply a CNOT operation with a control qubit $|0\rangle$ followed by a Hadamard operation as follows in Figure 2:

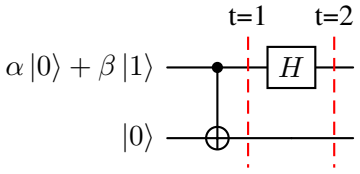


Figure 2: Alice's CNOT and Hadamard Operation

The intuition here is this: suppose Alice measures her qubit, the quantum state of the qubit is going to collapse to $|0\rangle$ or $|1\rangle$, which is clearly not what we want. By introducing the CNOT gate, the state is now entangled. One can realise that neither qubit in the entangled state contains the full information of the original qubit. Therefore at $t = 1$, the state of the quantum system, $|\psi_1\rangle$ is:

$$|\psi_1\rangle = (\alpha|0\rangle + \beta|1\rangle) \otimes |0\rangle = \alpha|00\rangle + \beta|11\rangle \tag{7.1}$$

Now, we want to remove the entanglement of the quantum state. Alice *could have* measured her qubit but that is not going to be helpful because with probability $|\alpha|^2$, her qubit is going to be $|0\rangle$ but then due to the entanglement, Bob's qubit is going to be $|0\rangle$ with the same probability. The same explanation applies for the state $|1\rangle$ with probability $|\beta|^2$. Applying the Hadamard operation on only Alice's qubit can remove that entanglement. We can see that at $t = 2$, the state of the quantum system $|\psi_2\rangle$ is

$$\begin{aligned} |\psi_1\rangle &\to |\psi_2\rangle \\ &= \alpha|+0\rangle + \beta|-1\rangle \\ &= \frac{\alpha}{\sqrt{2}}|00\rangle + \frac{\alpha}{\sqrt{2}}|10\rangle + \frac{\beta}{\sqrt{2}}|01\rangle - \frac{\beta}{\sqrt{2}}|11\rangle \end{aligned} \tag{7.2}$$

Now, when Alice does a partial measurement on her own qubit, the quantum state will collapse to

$$\begin{cases} \alpha|00\rangle + \beta|01\rangle & w.p \ \frac{|\alpha|^2+|\beta|^2}{2} = \frac{1}{2} \\ \alpha|10\rangle - \beta|11\rangle & w.p \ \frac{1}{2} \end{cases} \tag{7.3}$$

where the first case occurs when Alice observes $|0\rangle$ whereas the second case occurs when Alice observes $|1\rangle$. Denote the bit (not qubit anymore for Alice) that Alice observes to be b.

Alice then sends b to Bob (perhaps by telling Bob due to their close proximity in the lab). If $b = 0$, as mentioned above, the quantum state must have collapsed to $\alpha|00\rangle + \beta|01\rangle$. Bob's qubit would have been $\alpha|0\rangle + \beta|1\rangle$, which is exactly $|\psi\rangle$! However, if $b = 1$, then the quantum state must have collapsed to $\alpha|10\rangle - \beta|11\rangle$. Bob's qubit is $\alpha|0\rangle - \beta|1\rangle$. Bob simply has to perform a unitary operation, $U = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ to obtain $|\psi\rangle$!

Quantum Teleportation with an EPR Pair Now suppose that Alice and Bob are not near one another anymore but instead are really far apart! However, before Bob left Alice, both of them share an EPR pair. That is, each of them *share a qubit* of the EPR pair.

The first step is for Alice to entangle $|\psi\rangle$ with her own EPR pair using the CNOT and Hadamard operation as before. Bob's qubit (the EPR pair) remains untouched. The step involved in shown in Figure 3

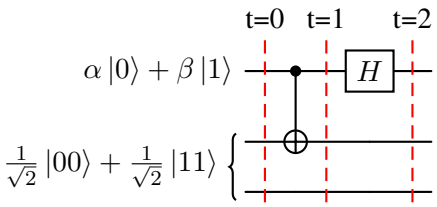


Figure 3: Entanglement of Alice's EPR pair with her qubit $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$

Let us go through the states for each of the time steps. At the beginning, $t = 0$, the joint state of the quantum circuit is given by $|\psi_0\rangle$:

$$|\psi_0\rangle = \frac{\alpha}{\sqrt{2}}|000\rangle + \frac{\alpha}{\sqrt{2}}|011\rangle + \frac{\beta}{\sqrt{2}}|100\rangle + \frac{\beta}{\sqrt{2}}|111\rangle \tag{7.4}$$

At $t = 1$, Alice has entangled her qubit with her own EPR pair to get a new joint state, $|\psi_1\rangle$:

$$|\psi_1\rangle = \frac{\alpha}{\sqrt{2}}|000\rangle + \frac{\alpha}{\sqrt{2}}|011\rangle + \frac{\beta}{\sqrt{2}}|110\rangle + \frac{\beta}{\sqrt{2}}|101\rangle \tag{7.5}$$

At $t = 2$, Alice has passed her qubit through a Hadamard gate which now results in $|\psi_2\rangle$

$$
\begin{aligned}
|\psi_1\rangle &= \frac{\alpha}{\sqrt{2}}|000\rangle + \frac{\alpha}{\sqrt{2}}|011\rangle + \frac{\beta}{\sqrt{2}}|110\rangle + \frac{\beta}{\sqrt{2}}|101\rangle \\
&\downarrow \\
|\psi_2\rangle &= \frac{\alpha}{\sqrt{2}}|+00\rangle + \frac{\alpha}{\sqrt{2}}|+11\rangle + \frac{\beta}{\sqrt{2}}|-10\rangle + \frac{\beta}{\sqrt{2}}|-01\rangle \\
&= \frac{\alpha}{\sqrt{2}}\left(\frac{1}{\sqrt{2}}(|000\rangle + |100\rangle)\right) + \frac{\alpha}{\sqrt{2}}\left(\frac{1}{\sqrt{2}}(|011\rangle + |111\rangle)\right) \\
&\quad + \frac{\beta}{\sqrt{2}}\left(\frac{1}{\sqrt{2}}(|010\rangle - |110\rangle)\right) + \frac{\beta}{\sqrt{2}}\left(\frac{1}{\sqrt{2}}(|001\rangle - |101\rangle)\right) \\
&= \frac{\alpha}{2}(|000\rangle + |100\rangle + |011\rangle + |111\rangle) + \frac{\beta}{2}(|010\rangle + |001\rangle - |110\rangle - |101\rangle)
\end{aligned}
\tag{7.6}
$$

The next crucial step for Alice is to do a partial measurement on the 2 qubits and *communicate her observations, b_0b_1, to Bob*. The measurement should result in an observation of one of 4 possible states: $|00\rangle, |01\rangle, |10\rangle, |11\rangle$. The effect of the measurements are detailed in Table 1.

Alice's observation, b_1b_0	State Collapses to	Bob's State
00	$\alpha\|000\rangle + \beta\|001\rangle$	$\|\psi\rangle = \alpha\|0\rangle + \beta\|1\rangle$
01	$\alpha\|011\rangle + \beta\|010\rangle$	$\alpha\|1\rangle + \beta\|0\rangle$
10	$\alpha\|100\rangle - \beta\|101\rangle$	$\alpha\|0\rangle - \beta\|1\rangle$
11	$\alpha\|111\rangle - \beta\|110\rangle$	$\alpha\|1\rangle - \beta\|0\rangle$

Table 1: Alice's observation and the effects of her measurement on the quantum circuit and hence, Bob's state.

Now let's turn to Bob. Suppose he receives Alice's 2-bit observation. He can then perform the following unitary operations according to the observed b_0b_1 *to directly recover* $|\psi\rangle$. The unitary operation U, which Bob has to perform, is as follows (note the order of operations):

$$
U = \begin{cases}
I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \text{ (equivalently: do nothing)} & \text{if } b_0b_1 = 00 \\
X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} & \text{if } b_0b_1 = 01 \\
Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} & \text{if } b_0b_1 = 10 \\
ZX = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} & \text{if } b_0b_1 = 11
\end{cases}
$$

We finally give the full diagram of the quantum circuit in Figure 4.

Remark 7.1 It might seem as if the teleported $|\psi\rangle$ is being cloned, which violates the Non-Cloning theorem. This is in fact not the case because Alice has lost her own qubit! The No-Cloning theorem is henceforth not violated. A useful sanity check is to realise that all the operations that we have done so far are linear.

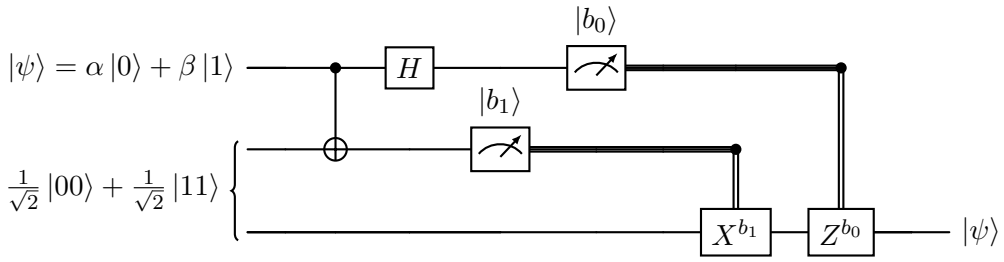


Figure 4: Quantum Teleportation Circuit with EPR Pair.

In conclusion, quantum teleportation demonstrates how quantum information can be transferred between distant locations without physically sending the particle that carries the state. By combining entanglement, measurement, and a bit of classical communication overhead, the protocol faithfully and securely reconstructs an unknown quantum state at a remote site (while necessarily destroying the original). This process highlights the power of entanglement as a resource and forms a foundational building block for quantum communication, quantum networks, and future quantum internet technologies.

Summary

Idea: Alice has a state $|\psi\rangle$ (whose amplitudes she doesn't even know). Through the quantum teleportation protocol she can send it to Bob using a shared $|EPR\rangle$ and two bits of classical communication.

Protocol: Starting from $|\psi\rangle|EPR\rangle$, Alice applies $H \circ CNOT$ on her two qubits to obtain

$$\frac{1}{2}[|00\rangle(\alpha|0\rangle + \beta|1\rangle) + |01\rangle(\alpha|1\rangle + \beta|0\rangle) + |10\rangle(\alpha|0\rangle - \beta|1\rangle) + |11\rangle(\alpha|1\rangle - \beta|0\rangle)].$$

Alice then measures her two qubits and obtains the results (b_0, b_1). Alice sends her results (comprising two bits of information) to Bob, who acts accordingly (apply $Z^{b_0}X^{b_1}$ on his qubit) to recover $|\psi\rangle$.

8 Classical Computation $\subseteq$ Quantum Computation

Quantum computers are arguably most interesting because of their potential in solving complex problems much faster than classical computers – if this were not the case then they would remain mostly a theoretical curiosity. Whether quantum $>$ classical – which can be captured mathematically rigorously via complexity statements like $BPP \subsetneq BQP$ or $NP \subsetneq QMA$ – remains to be established, with most in the community in the 'yes' camp. In this section we show that whatever a classical computer can do a quantum computer can as well, thus demonstrating that at the very least, quantum $\geq$ classical.

Because we have been talking about quantum computing in terms of gates and circuits, we begin by recasting classical computation in terms of (classical) gates and circuits as well, in contrast to the perhaps more well-known Turing machines formalism.

8.1 The Circuit Model of Computation

The circuit model of computation is a fundamental framework in theoretical computer science which most resembles the silicon chips making up modern computers. This model describes how computations are

performed using **circuits**, which are logical gates arranged in a network (specifically, a directed acyclic graph). The two most commonly used measures of complexity of a circuit C are its **size** $|C|$, which counts the number of *fundamental* gates in the circuit, and its **depth** $\text{dep}(C)$, which takes into account parallelism and counts the number of layers of (fundamental) gates comprising the circuit. These measures are each interesting in their own right and which one to use as figure of merit depends on context. Right now we shall mostly use the size $|C|$. By 'fundamental' above we refer to the gates belonging to any universal gate set. Recall that a universal gate set is one whose elements can be used to construct any Boolean function. The canonical example is $\{$AND, OR, NOT$\}$, but simpler sets also exist, such as $\{$AND, NOT$\}$ (OR can be constructed by de Morgan), or simply $\{$NAND$\}$.

A classic result by Cook and Karp states that any Turing machine M that computes an n-bit Boolean function in time $T(n)$ can be converted into a circuit family $\{C_n\}$ of size $O(T(n) \log T(n))$ which computes the same Boolean function. Roughly speaking this means that the circuit model of computation is just as powerful as the Turing machine model of computation since this only incurs a log overhead. For more details see Chapter 6 of [AB09].

We can further extend the above model to also allow for *probabilistic circuits* by introducing a FLIP gate, which outputs 0 or 1 with probability $1/2$ each.

Definition 8.1 A probabilistic circuit C computes

$$f : \{0,1\}^n \to \{0,1\}$$

if for all inputs x,

$$\Pr[C(x) = f(x)] \geq \frac{2}{3}.$$

The threshold $2/3$ was arbitrary, it could have been any number strictly greater than half. We can boost the threshold arbitrary close to 1 by parallel repetition.

8.2 Reversible Computing

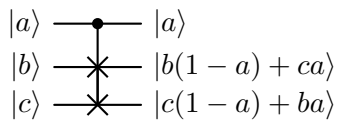
Recall that quantum gates are unitary, which means they are invertible/reversible. That is, given the output of a gate I can determine the input. This immediately presents a difficult if we want to convert classical gates to quantum gates – the gates such as AND or OR are clearly not reversible (the NOT gate on the other hand is reversible). For example, if we receive the output 0 from an AND gate, we cannot ascertain whether the input was $(0,0)$, $(0,1)$, or $(1,0)$.

To overcome this, we embed irreversible classical gates into reversible quantum gates by introducing additional input qubits called **ancillas** and allowing additional output qubits called **garbage**. Being able to simulate a universal set of logic gates with quantum gates then means we can simulate any classical computation quantumly. Of course we also have to make sure the overhead involved in these embeddings are not too high.

Below we present two three-qubit quantum gates which are universal for *classical* computing – the Toffoli (CCNOT) and Fredkin (CSWAP) gates. Alone they are not universal for quantum computing, but with an additional Hadamard they are. More precisely, this means that any arbitrary n-qubit unitary can be approximated arbitrarily closely using the $\{\text{CCNOT}, H\}$ or the $\{\text{CSWAP}, H\}$ gate sets.

Fredkin:

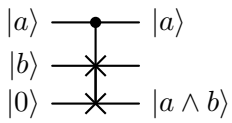
Consider the Fredkin/CSWAP gate on defined on basis states $|a\rangle$, $|b\rangle$ and $|c\rangle$ as such:



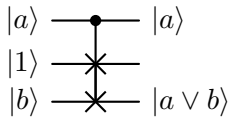
As a matrix this is

$$\text{CSWAP} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

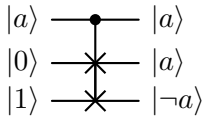
We can simulate AND:



OR:



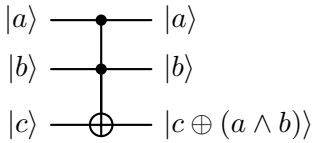
and NOT and COPY:



Thus, the Fredkin gate is universal for classical computation.

Toffoli:

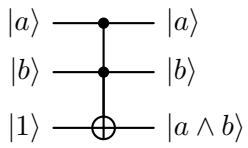
Consider the Toffoli/CCNOT gate defined on basis states $|a\rangle$, $|b\rangle$ and $|c\rangle$ as such:



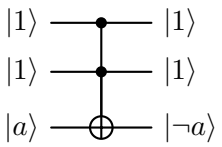
As a matrix this is

$$\text{CCNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}.$$

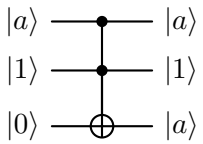
We can use also use it to simulate AND:



NOT:



and COPY:



We can use Toffoli to simulate OR as well, since $a \vee b = \neg(\neg a \wedge \neg b)$. Thus, the Toffoli gate is also universal for classical computation.

Either way, since we can use quantum gates to simulate a universal set of classical gates, any classical computation that can be done with classical gates can also be done with quantum gates. To summarize the above, we have:

Result 8.1 Given a function $f : \{0,1\}^n \to \{0,1\}^m$. Suppose we have a classical Boolean circuit C_f implementing f which requires k elementary gates. We can convert these gates into an $O(k)$-sized reversible circuit R_f comprising Toffoli/Fredkin gates, this also uses $O(k)$ ancillas. By copying $f(x)$ over to y using CNOTs, then uncomputing, we get the reversible circuit:

$$O_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$$

where we have omitted the ancillas. Roughly speaking, $O_f = R_f^{-1} CNOT^{\otimes n} R_f$, where R_f is a reversible implementation of C_f.

Uncomputing here just means applying R_f^{-1}, i.e. the inverse of R_f, see next section for details.

9 Basic Primitives in Quantum Computing

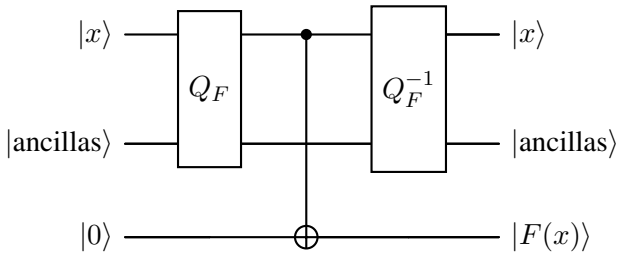
In this section we introduce several fundamental techniques that will be used repeatedly throughout this text. They are uncomputation (which allows us to remove garbage and reuse ancilla qubits), the phase-kickback method (which encodes function values into phases), and the Hadamard-Walsh transform, which is a fancy name for the parallel implementation of Hadamard gates.

9.1 Uncomputation

As discussed previously, a quantum implementation of a classical function typically produces additional intermediate data, usually referred to as garbage. Since quantum circuits are reversible, this garbage cannot be simply discarded. However, we would like to reuse ancillas which requires them to be reverted back to their original state.

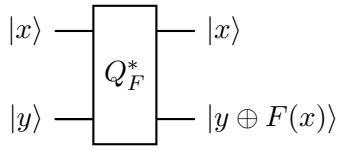
Let $F : \{0,1\}^n \to \{0,1\}$ and suppose we have a quantum circuit Q_F which, on input $|x\rangle$ and some ancilla qubits, produces $|F(x)\rangle$ together with some garbage.

The standard technique to remove garbage is the *compute–copy–uncompute* sequence. We first compute $F(x)$, then copy the result into a clean qubit using a CNOT, and finally apply the inverse circuit Q_F^{-1} to undo the computation and restore the ancillas. This process is commonly called **uncomputation**.



After applying Q_F^{-1} the garbage is removed and the ancillas are restored to their original state. We denote this entire implementation by $Q_F^* := Q_F^{-1} \circ \text{CNOT} \circ Q_F$.

More generally, for $F : \{0,1\}^n \to \{0,1\}^m$ and $y \in \{0,1\}^m$, the circuit Q_F^* implements



This form is essential because it avoids unwanted entanglement between the output and the ancillas and allows the ancilla qubits to be reused.

9.2 The Phase-Kickback Method

In many quantum algorithms we work with Boolean functions $F : \{0,1\}^n \to \{0,1\}$. It is often convenient to use the equivalent phase representation

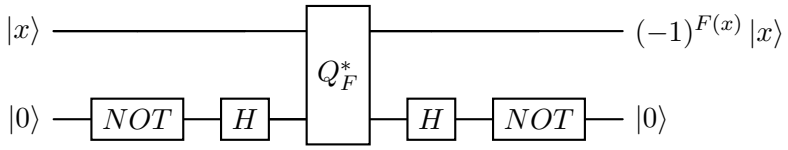
$$f(x) = (-1)^{F(x)} \in \{-1, 1\}.$$

A key technique for obtaining this phase representation is the **phase-kickback** trick. Consider the aforementioned Q_F^*. If we apply Q_F^* to $|x\rangle$ and $|-\rangle$ we obtain $\frac{1}{\sqrt{2}}(|F(x)\rangle - |\neg F(x)\rangle)$ as the second output which by closer inspection is the same as

$$(-1)^{F(x)} |x\rangle |-\rangle .$$

Thus the value of $F(x)$ is not stored in the target qubit; instead it appears as a phase factor on $|x\rangle$. This is the phase kickback effect.

Using standard gates, we can prepare $|-\rangle$ from $|0\rangle$ by applying a NOT gate followed by a Hadamard. This yields the circuit



We denote this entire quantum circuit by $Q_F^{\pm}$.

Why do we do this? Quantum algorithms exploit interference. Relative phases between basis states affect interference patterns. By encoding $F(x)$ as a phase rather than as a separate output bit, we allow the function values to influence interference directly.

9.3 The Hadamard-Walsh Transform

The operator $H^{\otimes n}$ denotes the parallel application of Hadamard gates to n qubits. This transformation, also known as the **Hadamard–Walsh transform**, maps

$$|0^n\rangle \mapsto \frac{1}{\sqrt{N}} \sum_{x \in \{0,1\}^n} |x\rangle , \qquad N = 2^n,$$

creating a uniform superposition over all bit strings. More generally,

$$H^{\otimes n} |x\rangle = \frac{1}{\sqrt{N}} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y} |y\rangle ,$$

where $x \cdot y$ is the bitwise inner product modulo 2.

Example. To build intuition, consider the case $|b_1, b_2\rangle$. Applying $H^{\otimes 2}$ gives

$$\frac{1}{2}\Big(|00\rangle + (-1)^{b_2}|01\rangle + (-1)^{b_1}|10\rangle + (-1)^{b_1+b_2}|11\rangle\Big).$$

Thus the output is a uniform superposition over all basis states, and the input bits b_1, b_2 only determine the *sign pattern* of the amplitudes. The phases are given by the parities $b_1x_1 + b_2x_2 \pmod 2$.

In general, the application of $H^{\otimes n}$ maps a computational basis state $|x\rangle$ to a vector with entries $\pm 1/\sqrt{2^n}$, where the s-th coordinate is given by

$$\frac{1}{\sqrt{2^n}}(-1)^{\sum_{i=1}^n x_i s_i \bmod 2}.$$

where $s \in \{0,1\}^n$ labels the computational basis states.

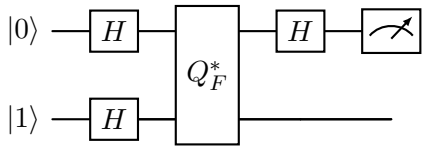
These vectors form the so-called χ-basis (the Fourier basis over $\mathbb{Z}_2^n$). From this perspective, the Hadamard–Walsh transform is simply the change of basis from the computational basis to the χ-basis.

10 Deutsch-Jozsa and Bernstein-Vazirani

First consider the baby version, 'Deutsch'. 'Deutsch-Jozsa' is a generalization of Deutsch to n-input bit functions.

10.1 Deutsch Algorithm

1. **Task:** The task is to determine whether a one-bit function $f : \{0,1\} \to \{0,1\}$ is constant or balanced. This function f is implemented through the oracle $Q_F^* |x,y\rangle = |x, y \oplus f(x)\rangle$. Because it acts bijectively on the basis states, it is a permutation and hence also a unitary.
2. **Protocol:** We start from the initialized state $|01\rangle$. We then apply the sequence $Q_F^* \circ H \otimes H$ to get (up to a global phase) the state $|\pm\rangle |-\rangle$, where it is $+$ if $f(0) = f(1)$ and $-$ if $f(0) \neq f(1)$. We discard the second qubit and apply a Hadamard to the first. This yields $|0\rangle \iff f(0) = f(1) \iff f(0) \oplus f(1) = 0$ (meaning the function is constant), or $|1\rangle \iff f(0) \neq f(1) \iff f(0) \oplus f(1) = 1$ (meaning the function is balanced). This entire process only requires one single Q_F^* call.



3. **Key Insight:** Note that the key driver of this algorithm is the phase-kickback mechanism $Q_F^* |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle$.

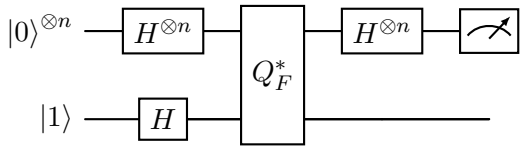
10.2 Deutsch-Jozsa Algorithm

Deutsch-Jozsa is a generalization of Deutsch to n-bit functions. It leverages massively parallel phase-kickback to evaluate the global property of the function.

1. **Task:** Determine whether an n-bit function $f : \{0,1\}^n \to \{0,1\}$ is constant or balanced.
2. **Mathematical fact:** Note that the Hadamard-Walsh transform on an n-qubit basis state yields $H^{\otimes n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_y (-1)^{x \cdot y} |y\rangle$.
3. **Protocol:** Start from the initial state $|0^n\rangle |1\rangle$. We apply the operation sequence $H^{\otimes n} \circ Q_F^* \circ H^{\otimes n} \otimes H$ to get the state:

$$\sum_{z \in \{0,1\}^n} \left(\frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x)+x \cdot z} \right) |z\rangle .$$

We then measure the working n qubits. Observe that the probability of measuring the all-zero state is $P(z = 0^n) = \left|\frac{1}{2^n}\sum_{x\in\{0,1\}^n}(-1)^{f(x)}\right|^2$. This probability equals 1 iff f is constant and equals 0 iff f is balanced. Again, this only requires one Q_F^* call.



4. **Classical comparison:** Classically, a deterministic worst-case evaluation needs to query more than half the space, i.e., $2^{n-1}+1 = O(2^n)$ queries. For a randomized algorithm: randomly pick k distinct inputs x_i and query to get $f(x_i)$; answer constant if $f(x_1) = \cdots = f(x_k)$, and balanced otherwise. If f was really constant, then the method works with certainty. However, if f was balanced, then the false-positive condition $f(x_1) = \cdots = f(x_k)$ occurs with probability $\frac{1}{2^{k-1}}$. So if our probability-error tolerance is ε, we would need $k = O(\log\frac{1}{\varepsilon})$ queries.

10.3 Bernstein-Vazirani Algorithm

Related to Deutsch-Jozsa is Bernstein-Vazirani, albeit with a slightly different end goal. The structure of the algorithm is very similar.

1. **Task:** Given a function $f : \{0,1\}^n \to \{0,1\}$ such that $f(x) = x \cdot s$ for some unknown secret string s, the goal is to find s.
2. **Classical Bound:** Classically, the most efficient method is to probe the function bit by bit to obtain $f(1,0,\dots) = s_1, f(0,1,\dots) = s_2, \dots$. This strictly requires n queries.
3. **Quantum Protocol:** Quantumly, the algorithm is the exact same as Deutsch-Jozsa: we apply the sequence $H^{\otimes n} \circ Q_F^* \circ H^{\otimes n} \otimes H$. Miraculously, measuring the query register produces the final output $|s\rangle$ with absolute certainty!
4. **Explanation:** Ok, actually there is nothing miraculous - it's simply because after applying the oracle query, phase kickback leaves us with the state $\frac{1}{\sqrt{2^n}}\sum_x(-1)^{x\cdot s}|x\rangle$. This expression is exactly the mathematical expansion of $H^{\otimes n}|s\rangle$. And since Hadamards are self-inverse ($H^2 = I$), applying the final layer of Hadamards exactly recovers the string $|s\rangle$.

11 Quantum Fourier Transform(s)

Before we begin let us recall the notion of a Fourier transform, first conceived of by Joseph Fourier in his analysis of heat flow. The core idea is to decompose an arbitrary function in terms of simpler, well-behaved functions, historically called 'frequency functions' because in that context these simpler functions were sine/cosine wave functions parametrized by frequencies. More specifically, 'well-behaved' here refers to invariance under the translation operation $T_a f(x) := f(x + a)$. The translation-invariant functions are the eigenfunctions of T_a for all $a \in \mathbb{R}$. In Joseph Fourier's case these were the frequency basis functions $\chi_\xi(x) := e^{2\pi i \xi x}$, and the Fourier-transformed function $\hat{f}$ is given as a linear combination of these:

$$\hat{f}(\xi) = \int_\infty^\infty f(x) e^{-2\pi i \xi x} dx.$$

The integral here can be thought of as a linear combination. If you are willing to accept the vector notation

$$|f\rangle = \int_{-\infty}^{\infty} f(x) \, |x\rangle \, dx$$

and

$$|\xi\rangle = \int_{-\infty}^{\infty} \chi_\xi(x) \, |x\rangle \, dx$$

then taking their inner product (which is defined as $\langle f|g\rangle := \int_\infty^\infty f(x)\overline{g(x)}dx$ for complex-valued functions) one can check that $\langle \xi|f\rangle = \hat{f}(\xi)$. Note that we have used the Dirac delta function, which is the infinite-dimensional variant of the Kronecker delta: $\langle y|x\rangle = \delta(x - y)$. The key takeaway is $\langle x|f\rangle = f(x)$ and $\langle \xi|f\rangle = \hat{f}(\xi)$, thus $f(x)$ and $\hat{f}(\xi)$ are really simply different ways of representing f, with respect to different bases. Also note that technically speaking, the Fourier transform is a function $\mathcal{F}$ taking as input a function f and outputting another function $\hat{f}$, i.e.

$$\mathcal{F}(f) = \hat{f}.$$

It has become somewhat customary to refer to $\hat{f}$ as the Fourier transform of f however, because of convenience, and we too shall often do so. In practice this doesn't cause much confusion but just be aware of the difference.

Before we proceed further let us point out a way of viewing things that is pervasive throughout discrete mathematics: *functions and vectors are the same objects*. Specifically, what this refers to is the

Fact 11.1 (Function-Vector Equivalence) Let S be a finite set and G be a field. Then functions f with domain S and codomain G are the same as $|S|$-sized column/row vectors v taking values in G, through the correspondence

$$f(i) \leftrightarrow v_i.$$

Here since S is finite we can let it be $[N]$ for some positive integer N, and in practice G is usually a number system such as the real numbers/complex numbers/finite fields or subsets thereof.

There is nothing deep about this statement, simply it means if f is defined over a finite set of elements we can enumerate all its values (in some arbitrarily chosen order), then compile it into a vector. In fact this could be generalized to infinite-sized domains, with additional technical considerations of course. The reason we mention this here is because the Fourier transform is frequently stated for both functions and vectors, and there is a potential misunderstanding that these are different things, i.e. there are notions of Fourier transforms for functions, and also for vectors. These are really the same thing.

With this equivalence in mind, if we write f as a column vector, then because the Fourier transform/operator is a linear function, it admits a matrix representation, such as the $H^{\otimes n}$ and DFT operators we shall see below. These matrices are also referred to as Fourier transforms. Hopefully this isn't too confusing. Just bear in mind that functions $\leftrightarrow$ vectors and linear functions of functions $\leftrightarrow$ matrices and you'll be fine.

Back to the Fourier transform. The Fourier transform/decomposition was subsequently generalized to spaces beyond $\mathbb{R}^n$, and the study of this theory in the modern context is also known as harmonic analysis, which has deep links to many other fields of mathematics, most notably PDEs and representation theory. What we called 'frequencies' also came to adopt fancier names, such as 'harmonics', or 'characters' (in the representation theory context). In particular, the theory of Fourier/Harmonic analysis over *finite, abelian* groups is especially well-developed. As it turns out, in computer science and this course in particular we are most interested in the set $[N]$, where $N = 2^n$ for some n. This set is quite naturally called the Boolean hypercube.

Now note that there are two distinct abelian group operations $[N]$ admits. First, if $N = 2^n$ then representing the numbers in binary we can equip $[N]$ with the bitwise addition modulo 2 operation. Second, for any positive integer N (not just those which are powers of two), $[N]$ can be equipped with the addition modulo N operation. These are two *distinct* group operations defined on the *same* underlying set, and give rise to different Fourier transforms. The Hadamard-Walsh transform is precisely the Fourier transform with respect to the first group operation, where we now call the group $\mathbb{Z}_2^n$. The 'conventional' Fourier transform, also called the discrete Fourier transform is the Fourier transform with respect to the second group operation, where we now call the group $\mathbb{Z}_{2^n}$. This is the one people (including the standard textbook in quantum information and computation, [NC10]) usually refer to when they say 'quantum Fourier transform' (which technically refers to the quantum implementation of the DFT).

We first revisit the Hadamard-Walsh transform, before delving into the QFT. In both cases we shall proceed along the following structural flow:

1. Specify the group.
2. Identify the translation operators $T_a f(x) := f(x + a)$. Note that the presence of the $+$ symbol here indicates that translation is a *group-dependent* notion.
3. Work out the form of the characters, which are the simultaneous eigenfunctions of all translation operators.
4. Work out the form of the corresponding Fourier transform matrix, which is the change-of-basis matrix between the canonical basis and the character basis.

5. Work out the Fourier coefficients, which are the coefficients of the function with respect to the character basis.

11.1 Hadamard-Walsh revisited

Group: Group is designated $\mathbb{Z}_2^n$. Underlying set is $\{0,1\}^n$. Group operation is bitwise addition modulo 2. Functions on this group we denote by $f : \mathbb{Z}_2^n \mapsto \mathbb{C}$, which we can also write as $|f\rangle = \frac{1}{\sqrt{N}} \sum_x f(x) |x\rangle$, where $|x\rangle$ is the 'canonical basis', which in column vector form has one '1' entry and zeros elsewhere.

Translation Operators: Translation operators take in functions and output shifted functions. Here for each bitstring $a \in \mathbb{Z}_2^n$ we have a translation operator

$$T_a f(x) := f(x + a).$$

The '+' for $\mathbb{Z}_2^n$ is often written as '$\oplus$'. Note that translation operators are unitary, $T_a^\dagger = T_{-a}$, so their eigenvalues all have modulus 1. T_a and T_b commute for all a, b, thus they share simultaneous eigenvectors.

Characters: These are the simultaneous eigenvectors $\chi(x)$ of all the T_a's:

$$T_a \chi(x) = \lambda(a)\chi(x).$$

Note that the eigenvalues generally depend on the shift amount a. Just as we had $|f\rangle = \frac{1}{\sqrt{N}} \sum_x f(x) |x\rangle$, here we have $|\chi\rangle = \frac{1}{\sqrt{N}} \sum_x \chi(x) |x\rangle$.

Let us now determine the form of $\chi(x)$. First, $\chi(x + a) =: T_a\chi(x) = \lambda(a)\chi(x)$ is to hold for all $x, a \in \mathbb{Z}_2^n$. Setting $x = 0$ in particular yields $\chi(a) = \lambda(a)\chi(0)$, which we can just as well write $\chi(x) = \lambda(x)\chi(0)$ for all bitstrings x. Note that $\chi(0) \neq 0$, otherwise $\chi(x) = 0$ is the zero vector (which by definition disqualifies it from being an eigenvector). Now consider the bitstring sum $x + a$ for any x, a. On one hand we immediately have $\chi(x + a) = \lambda(x + a)\chi(0)$; on the other hand we also have $\chi(x + a) = \lambda(a)\chi(x) = \lambda(a)\lambda(x)\chi(0)$, thus yielding the eigenvalue identity $\lambda(x + a) = \lambda(x)\lambda(a)$.

Recall now that $\chi(x) = \lambda(x)\chi(0)$. We make the *convention* that $\chi(0) = 1$. Why is this a convention? Recall that if $Av = \lambda v$ is an eigenvalue equation, the scaled eigenvector cv satisfies the same eigenvalue equation $A(cv) = \lambda(cv)$ as well. In practice one usually sets a convention for the eigenvectors, for instance that they be normalized: $\|v\|_2 = 1$. Our convention here $\chi(0) = 1$ exactly does this. Why? Because

$$\langle\chi|\chi\rangle = \frac{1}{N} \sum_x |\chi(x)|^2 = \frac{1}{N} |\chi(0)|^2 \sum_x |\lambda(x)|^2 = |\chi(0)|^2$$

where in the last inequality we have invoked the unit modulus property of translator eigenvalues. Thus we see that setting $\chi(0) = 1$ yields normalized eigenvectors.

With this, we have the character identity

$$\chi(x + a) = \chi(x)\chi(a).$$

That is, χ is an abelian group homomorphism from $\mathbb{Z}_2^n$ to $\mathbb{C}^\times$. $\mathbb{C}^\times$ is the *multiplicative group of complex numbers* – the underlying set is $\mathbb{C}\backslash\{0\}$ and the group operation is multiplication (not addition).

Let us proceed further. Under the bitwise addition modulo 2 operation $x + x = 0$ for all bitstrings x. Thus $\chi(0) = \chi(x + x) = \chi(x)^2$. Since this holds for all x, including the all-zeros string, we deduce that $\chi(x) = \pm 1$ for all x. How many (independent) functions χ's satisfy this? Well consider for simplicity the $n = 1$ case: we have $\chi(0) = 1$ and $\chi(1) = \pm 1$, two options. We can write this succinctly as $\chi_s(x) = (-1)^{sx}$, i.e. the characters are indexed by s. In the $n = 1$ case $s = 0, 1$. For general n the logic is similar, just that the indices are now bitstrings s. The characters in this general case are given by

$$\chi_s(x) = (-1)^{s \cdot x}$$

and

$$|\chi_s\rangle = \frac{1}{\sqrt{N}} \sum_x (-1)^{s \cdot x} |x\rangle .$$

Please verify the orthonormality condition yourself: $\langle \chi_t | \chi_s \rangle = \delta_{st}$.

Fourier Transform: The Fourier transform matrix is simply defined to be the change-of-basis matrix from $|s\rangle$ to $|\chi_s\rangle$, i.e. the most general form is

$$\text{FT} |s\rangle := |\chi_s\rangle .$$

In $\mathbb{Z}_2^n$, bra-ing both sides by $\langle t|$ yields

$$\langle t | \text{FT}(\mathbb{Z}_2^n) | s \rangle = \frac{1}{\sqrt{N}} (-1)^{s \cdot t},$$

thus in $\mathbb{Z}_2^n$

$$\text{FT}(\mathbb{Z}_2^n) = H^{\otimes n} = \frac{1}{\sqrt{N}} \sum_{x,y \in \mathbb{Z}_2^n} (-1)^{x \cdot y} |x\rangle\langle y| .$$

Please remember that here $N = 2^n$.

Fourier Coefficients: Finally, the Fourier coefficients are simply defined to be $\hat{f}(s) := \langle \chi_s | f \rangle$. Here it is

$$\hat{f}(s) = \frac{1}{N} \sum_{x \in \mathbb{Z}_2^n} (-1)^{s \cdot x} f(x).$$

11.2 Quantum Fourier Transform

The Quantum Fourier Transform (henceforth simply QFT) is the quantum analogue of the classical discrete Fourier transform. It is a fundamental subroutine in many quantum algorithms, including Shor's algorithm for factoring, owing to its ability to extract periodicity efficiently.

Group: Group is designated $\mathbb{Z}_N = \mathbb{Z}_{2^n}$. Underlying set is still the same, $\{0, 1\}^n$. Group operation is addition modulo N. Functions on this group we denote by $f : \mathbb{Z}_N \mapsto \mathbb{C}$, which we can also write as $|f\rangle = \frac{1}{\sqrt{N}} \sum_x f(x) |x\rangle$.

Translation Operators: Translation operators take in functions and output shifted functions. Here for each number $a \in \mathbb{Z}_N$ we have a translation operator

$$T_a f(x) := f(x + a).$$

Note that translation operators are unitary, $T_a^\dagger = T_{-a}$, so their eigenvalues all have modulus 1. T_a and T_b commute for all a, b, thus they share simultaneous eigenvectors.

Characters: Again these are the simultaneous eigenvectors $\chi(x)$ of all the T_a's:

$$T_a \chi(x) = \lambda(a)\chi(x).$$

Let us now determine the form of $\chi(x)$ for $\mathbb{Z}_N$. The argument sequence concerning the characters $\chi(x)/\left|\chi\right\rangle$ we saw above can be replicated here, mutatis mutandis, up until and including the fact

$$\chi(x + a) = \chi(x)\chi(a).$$

Once again χ is an abelian group homomorphism from $\mathbb{Z}_N$ to $\mathbb{C}^\times$.

Now we explore the analytic form of $\chi(x)$ for $\mathbb{Z}_N$, which differs from that in the $\mathbb{Z}_2^n$ case. Since 1 is the generator of $\mathbb{Z}_N$, we have $\chi(x) = \chi(1)^x$ for all x. In particular, letting $x = N$ yields $\chi(1)^N = 1$. We thus deduce that there are N possible choices for $\chi(1)$, and index them by $s = 0, \ldots, N - 1$:

$$\chi_s(1) = e^{2\pi i s/N}.$$

Once we fix an s this defines the rest of the $\chi_s(x)$'s. This yields

$$\chi_s(x) = e^{2\pi i s x/N}.$$

The corresponding kets are now

$$\left|\chi_s\right\rangle = \frac{1}{\sqrt{N}} \sum_x e^{2\pi i s x/N} \left|x\right\rangle.$$

Please verify the orthonormality condition yourself: $\langle\chi_t|\chi_s\rangle = \delta_{st}$. In this case this simply relies on summing through a geometric sequence.

Fourier Transform: The Fourier transform matrix is the change-of-basis matrix from $|s\rangle$ to $|\chi_s\rangle$, i.e. the most general form is

$$\mathrm{FT}\left|s\right\rangle := \left|\chi_s\right\rangle.$$

In $\mathbb{Z}_N$, bra-ing both sides by $\langle t|$ yields

$$\langle t|\mathrm{FT}(\mathbb{Z}_N)|s\rangle = \frac{1}{\sqrt{N}} e^{2\pi i s t/N},$$

thus in $\mathbb{Z}_N$

$$\mathrm{FT}(\mathbb{Z}_N) = \mathsf{DFT} = \frac{1}{\sqrt{N}} \sum_{x,y \in \mathbb{Z}_N} e^{2\pi i x y/N} \left|x\right\rangle\!\left\langle y\right|.$$

Please remember that here $N = 2^n$.

Fourier Coefficients: Finally, the Fourier coefficients are simply defined to be $\hat{f}(s) := \langle \chi_s | f \rangle$. Here it is

$$\hat{f}(s) = \frac{1}{N} \sum_{x \in \mathbb{Z}_N} e^{-2\pi i s x/N} f(x).$$

Okay. Now strictly speaking everything above is called the Discrete Fourier Transform, DFT. The *quantum* Fourier transform simply refers to implementing the DFT quantumly. Nothing above so far has anything to do with quantum, apart from the cultural appropriation of bra-ket notation. Right now, we enter quantum. For the group $\mathbb{Z}_{2^n}$ the quantum implementation was simply by parallel Hadamarding, so there wasn't too much to speak of. For the DFT things are a bit trickier.

1. **Binary Representation:** To express the QFT elegantly, it is convenient to utilize the binary representation of numbers. For an integer $\alpha \geq 1$, its binary expansion is written as:

   $$\alpha = \alpha_1 \dots \alpha_n = \alpha_1 2^{n-1} + \alpha_2 2^{n-2} + \cdots + \alpha_n 2^0$$

   where each $\alpha_i \in \{0, 1\}$. Similarly, we can represent fractional numbers $0 \leq \alpha < 1$ using binary fractions:

   $$\alpha = 0.\alpha_1 \dots \alpha_n = \alpha_1 2^{-1} + \alpha_2 2^{-2} + \cdots + \alpha_n 2^{-n}$$

   Note the direct algebraic relationship: if we divide an integer $\alpha = \alpha_1 \dots \alpha_n$ by 2^n, we simply shift the binary point to the left n times, yielding the fraction $\alpha/2^n = 0.\alpha_1 \dots \alpha_n$.

2. **DFT and QFT:** Recall:

   $$\mathsf{DFT} = \frac{1}{\sqrt{N}} \sum_{j,k=0}^{N-1} e^{2\pi i j k/N} \left| j \right\rangle\!\left\langle k \right|.$$

   The Quantum Fourier Transform implements the exact same linear transformation, but it acts on the probability amplitudes of a quantum state. For a given computational basis state $|j\rangle$, the QFT applies the following superposition:

   $$\mathsf{QFT} \left| j \right\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i j k/N} \left| k \right\rangle.$$

   By leveraging the binary fractional representation discussed in the previous point, we can factor this sum into a tensor product of n individual qubits. In binary, the state becomes:

   $$\mathsf{QFT} \left| j_1 \dots j_n \right\rangle = \frac{1}{\sqrt{2^n}} \left(|0\rangle + e^{2\pi i 0.j_n} |1\rangle \right) \left(|0\rangle + e^{2\pi i 0.j_{n-1} j_n} |1\rangle \right) \cdots \left(|0\rangle + e^{2\pi i 0.j_1 j_2 \dots j_n} |1\rangle \right).$$

   This product form is remarkably useful as it reveals that the QFT leaves the qubits in a completely unentangled state (a product state), provided the input was a basis state.

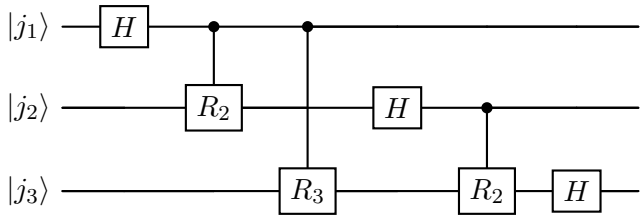
3. **Derivation of the Binary Representation of QFT:** We can mathematically derive the tensor product form from the standard QFT definition by expanding the index k into its binary representation $k = k_1 k_2 \dots k_n = \sum_{l=1}^{n} k_l 2^{n-l}$:

   $$\mathsf{QFT} \left| j_1 \dots j_n \right\rangle = \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{\frac{2\pi i j k}{2^n}} \left| k \right\rangle$$

$$
\begin{aligned}
&= \frac{1}{\sqrt{2^n}} \sum_{k_1,\ldots k_n=0}^{1} e^{2\pi i j(\sum_{l=1}^{n} k_l 2^{-l})} \left|k_1 \ldots k_n\right\rangle \\
&= \frac{1}{\sqrt{2^n}} \sum_{k_1,\ldots k_n=0}^{1} \bigotimes_{l=1}^{n} e^{2\pi i j k_l 2^{-l}} \left|k_l\right\rangle \\
&= \frac{1}{\sqrt{2^n}} \bigotimes_{l=1}^{n} \left[\sum_{k_l=0}^{1} e^{2\pi i j k_l 2^{-l}} \left|k_l\right\rangle\right] \\
&= \frac{1}{\sqrt{2^n}} \bigotimes_{l=1}^{n} \left[\left|0\right\rangle + e^{2\pi i j 2^{-l}} \left|1\right\rangle\right] \\
&= \frac{1}{\sqrt{2^n}} \left(\left|0\right\rangle + e^{2\pi i 0.j_n} \left|1\right\rangle\right) \left(\left|0\right\rangle + e^{2\pi i 0.j_{n-1} j_n} \left|1\right\rangle\right) \cdots \left(\left|0\right\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n} \left|1\right\rangle\right).
\end{aligned}
$$

This derivation gives the core intuition for constructing the efficient quantum circuit. We define the phase rotation gate $R_k := \text{diag}(1, e^{\frac{2\pi i}{2^k}})$. To generate the fractions $0.j_l \ldots j_n$ in the phases, we first apply a Hadamard gate to put the qubit in a superposition $\left|0\right\rangle + e^{2\pi i 0.j_l} \left|1\right\rangle$, and then apply controlled-R_k gates from the lower significant qubits to systematically add the smaller binary fractions to the phase.

A 3-qubit QFT circuit, comprising the H and controlled-R_k gates, is illustrated below (note that the output order of qubits is reversed and generally requires SWAP gates at the end, which are often omitted for simplicity):



4. **Algorithm Speedup and Caveats:** The efficiency gains from the QFT are significant when viewed in terms of n, the number of bits (where $N = 2^n$). The classical FFT takes $\Theta(N \log N) = \Theta(n2^n)$ operations. In contrast, analyzing the circuit above, the i-th qubit requires one Hadamard and $n - i$ controlled-phase gates. Summing this over n qubits gives $\frac{n(n+1)}{2}$ gates. Thus, the QFT takes $\Theta(n^2) = \Theta(\log^2 N)$ gates, an exponential speedup over the classical FFT.

   Caveat: The Fourier-transformed output state $\mathsf{QFT}\left|x\right\rangle$ is not directly analogous to having the vector $\text{DFT}(x)$ in memory. The resulting frequency components are encoded purely in the *amplitudes* of the quantum state. To actually extract or learn these values, we must perform measurements. Due to the probabilistic nature of quantum measurement, extracting the full set of amplitudes requires executing the circuit exponentially many times, which eliminates the speedup if the full frequency spectrum is the goal. The QFT's true utility lies as an intermediate step where global properties (like periods) can be extracted without having to inspect every amplitude.

11.3 Comparison between Hadamard-Walsh and QFT

Now you have seen both Fourier transforms. You have seen how different group operations give rise to different translation operators, which give rise to different characters, which give rise to different Fourier transforms. Please try to grasp the underlying logic and not rely too much on memorization; the indices here alone can kill. It is quite sufficient to remember the character forms $\chi_s(x) = (-1)^{s \cdot x}$/$\chi_s(x) = e^{2\pi i s x/N}$; everything else follows naturally if you understand what they fundamentally are.

Alright. Here is a table comparing Hadamard-Walsh and QFT, in case you ever need it.

Feature	**Hadamard–Walsh**	**DFT / QFT over** $\mathbb{Z}_N$ (often $N = 2^n$)
Underlying Set	$\{0,1\}^n$	$\{0,1,\ldots,N-1\}$
Group Operation	Bitwise addition mod 2	Addition mod N
Group	$\mathbb{Z}_2^n$	$\mathbb{Z}_N$
Characters χ	$\chi_s(x) = (-1)^{s \cdot x}$ $\lvert\chi_s\rangle = \frac{1}{\sqrt{N}} \sum_{x \in \mathbb{Z}_2^n} (-1)^{s \cdot x} \lvert x\rangle$	$\chi_s(x) = e^{2\pi i s x/N}$ $\lvert\chi_s\rangle = \frac{1}{\sqrt{N}} \sum_{x \in \mathbb{Z}_N} e^{2\pi i s x/N} \lvert x\rangle$
Transform Function Form Vector Form	 $\hat{f}(s) = \frac{1}{N} \sum_{x \in \mathbb{Z}_2^n} (-1)^{s \cdot x} f(x)$ $\beta_s = \frac{1}{N} \sum_{x \in \mathbb{Z}_2^n} (-1)^{s \cdot x} \alpha_x$	 $\hat{f}(s) = \frac{1}{N} \sum_{x \in \mathbb{Z}_N} e^{-2\pi i s x/N} f(x)$ $\beta_s = \frac{1}{N} \sum_{x \in \mathbb{Z}_N} e^{-2\pi i s x/N} \alpha_x$
Operator/Matrix Representation	$H^{\otimes n} = \frac{1}{\sqrt{N}} \sum_{x,y \in \mathbb{Z}_2^n} (-1)^{x \cdot y} \lvert x\rangle\langle y\rvert$	$\text{DFT} = \frac{1}{\sqrt{N}} \sum_{x,y \in \mathbb{Z}_N} e^{2\pi i x y/N} \lvert x\rangle\langle y\rvert$

And here's the one for the OG Fourier transform:

Feature	**Classical Fourier Transform**
Underlying Set	$\mathbb{R}$
Group Operation	Addition
Group	$(\mathbb{R}, +)$
Characters χ	$\chi_\xi(x) = e^{2\pi i \xi x},\ \xi \in \mathbb{R}$
Transform	$\widehat{f}(\xi) = \int_{\mathbb{R}} f(x) e^{-2\pi i \xi x}\, dx$

11.4 Appendix: The Fast Fourier Transform

Fast Fourier Transform (Cooley-Tukey Algorithm): Classically, computing the DFT directly from the definition requires $O(N^2)$ operations. The Cooley-Tukey Fast Fourier Transform (FFT) algorithm reduces this drastically. The algorithm proceeds as follows:

Start by splitting the DFT sum (absorb $1/\sqrt{N}$ into the x_j for brevity) into its even and odd indexed terms:

$$\begin{aligned} y_k &= \sum_{j=0}^{N-1} e^{\frac{2\pi i j k}{N}} x_j \\ &= \sum_{j=0}^{N/2-1} e^{\frac{2\pi i (2j) k}{N}} x_{2j} + \sum_{j=0}^{N/2-1} e^{\frac{2\pi i (2j+1) k}{N}} x_{2j+1} \end{aligned}$$

$$= \underbrace{\sum_{j=0}^{N/2-1} e^{\frac{2\pi ijk}{N/2}} x_{2j}}_{E_k} + e^{\frac{2\pi ik}{N}} \underbrace{\sum_{j=0}^{N/2-1} e^{\frac{2\pi ijk}{N/2}} x_{2j+1}}_{O_k}.$$

Notice that E_k and O_k are just smaller DFTs of size $N/2$. The key breakthrough of Cooley-Tukey is noticing the symmetry in the roots of unity, specifically $e^{\frac{2\pi i(k+N/2)}{N}} = -e^{\frac{2\pi ik}{N}}$, which allows us to write:

$$y_{k+\frac{N}{2}} = E_k - e^{\frac{2\pi ik}{N}} O_k.$$

This symmetry means we only have to compute the first half of the sequence (y_k from $k = 0$ until $k = N/2-1$) to get the second half for free. By recursively breaking this up into smaller DFTs until reaching the base case of $N = 2$, we continually reuse the results of intermediate computations. We can evaluate the total complexity by drawing the binary tree expansion of this procedure: at each level l, there are 2^l E_k/O_k terms, and for each term, we have $N/2^l$ values of k. Each E_k/O_k term requires combining its children nodes, taking $O(1)$ operations. With $O(N)$ operations per level and $\log_2 N$ levels in the tree, the total classical time complexity evaluates to $O(N \log N)$.

Instructions

- Try to leave the preamble unchanged, only make changes to `main.tex` and `references.bib`. Add images to the `images` folder.
- Share one (zipped) directory preserving the directory structure on Canvas.
- Prepare organized notes by elaborating on the broad items provided in different sections below based on the lecture material completing any missing details from lectures.
- Feel free to modify structure and/or order of contents as you see fit; the below is just a guideline.

12 Simon's Problem and Algorithm

1. Problem: given $f : \{0,1\}^n \to \{0,1\}^n$ s.t. $f(x) = f(y) \iff x = y \oplus s$, find s.
2. Classical Case
   (a) A randomized algorithm: randomly pick k (TBD) distinct inputs x_i and query to get $f(x_i)$. If there is no collision, answer $s = 0^n$. If there is a collision $f(x_i) = f(x_j)$, then answer $s = x_i \oplus x_j$.
   (b) Analysis: If actually $s = 0^n$ then the algorithm works with certainty. If actually $s \neq 0^n$ then we might answer wrongly if there is no observed collisions. Let the probability that the samples have no collision after $k \geq 2$ queries be $P(k)$. Suppose after $j-1$ queries there is no matching. The probability that on the jth, i.e. next query there is still no collision with one of the previous queries is then $1 - \frac{j-1}{2^n-(j-1)}$. Thus after k rounds,

$$\begin{aligned} P(k) &= \prod_{j=2}^{k} \left[1 - \frac{j-1}{2^n-(j-1)}\right] \\ &\geq 1 - \sum_{j=2}^{k} \left[\frac{j-1}{2^n-(k-1)}\right] \\ &\geq 1 - \frac{\Theta(k^2)}{2^n - \Theta(k)}. \end{aligned}$$

   Remember that we *do* want a collision! So we want $P(k)$ to be small, say ε (equivalently, coll. prob. is $1-\varepsilon$). Then $P(k) < \varepsilon \implies k \in \Omega(2^{n/2})$. Here in the first $\geq$ we used $(1-a)(1-b) \geq 1-(a+b)$ for $0 \leq a, b \leq 1$. Also note that we can assume from the outset that $k \leq 2^{n/2}$ because if $k > 2^{n/2}$ then there has to be a collision.
   (c) Simon also showed *any* randomized algorithm needs $k = \Omega(2^{n/2})$ queries (proof is similar to the one above).

3. Simon's Quantum Algorithm

   (a) Quantum part: Start with n work qubits and n ancilla qubits $|0^n\rangle|0^n\rangle$. Apply $(H^{\otimes n} \otimes I)O_f(H^{\otimes n} \otimes I)$ then measure the work qubits.

   (b) Classical postprocessing: repeat $n-1$ times to get $n-1$ strings y_i all satisfying $y_i \cdot s = 0$, where y_i are linearly independent with positive probability. If lin. ind., use Gaussian elimination to solve system, giving two candidates 0^n and $s' \neq 0^n$.

   (c) Test: if $f(0^n) = f(s')$, answer $s = s'$; otherwise $f(0^n) \neq f(s')$, answer $s = 0^n$.

   (d) Boosting: repeat entire process k times to boost prob. of obtaining lin. ind. y_i. (LI can be easily checked.) If for any run $f(0^n) = f(s')$, answer $s = s'$; otherwise if for all runs $f(0^n) \neq f(s')$, answer $s = 0^n$.

4. Analysis of Simon's algorithm

   (a) Quantum: after applying the unitaries the state is

$$|\psi\rangle = \sum_y |y\rangle \left(\frac{1}{2^n} \sum_x (-1)^{x \cdot y} |f(x)\rangle \right).$$

   If $s = 0^n$, f is injective so $p(y) = \frac{1}{2^n}$ for all y. If $s \neq 0^n$, f is two-to-one, so if $z \in \text{range}(f)$ then $f(x_z) = f(x'_z) = z$ for $x_z = x'_z \oplus s$. Then $p(y) = \frac{1}{2^{n-1}}$ if $y \cdot s = 0$, and $p(y) = 0$ if $y \cdot s = 1$.

   (b) Postprocessing: suppose we have lin. ind. $\{y_i\}_{i=1}^{n-1}$. GE gives the two possible solutions: $0^n, s' \neq 0^n$. Suppose actually $s = 0^n$. Then $f(0^n) \neq f(s')$, so our answer $s = 0^n$ is always correct. Suppose actually $s \neq 0^n$. Then $f(s) = f(0^n) = f(s')$, so our answer $s' = s$ is always correct.

   (c) Probability of linear independence: given m LI strings $y_1, \ldots, y_m$, spanning subspace is size 2^m. Prob of y_{m+1} being LI to this is $\frac{2^n - 2^m}{2^n}$. So prob of $n-1$ LI strings is $P = (1 - \frac{2^1}{2^n}) \ldots (1 - \frac{2^{n-2}}{2^n}) \geq 1 - \sum_{i=1}^{n-2} \frac{1}{2^i} > \frac{1}{2}$. So prob. of *no LI* even after running k times is at most $(1 - \frac{1}{2})^k < e^{-k/2}$. So prob. of *at least one LI* after running k times is at least $1 - e^{-k/2}$.

KK: Today Divesh will continue discussing the Fourier transform over Z_N, and also talk about Simon's algorithm over Z_N. These closely parallel the FT and Simon over Z_2^n, recall that the underlying sets are the same, but group operations are different. You can refer to the Lec6 final notes as a guide. If there's time will also discuss the connection between period finding and factoring.

13 Simon over Z_N

References

[AB09] Sanjeev Arora and Boaz Barak. *Computational complexity: a modern approach*. Cambridge University Press, 2009. {34}

[AKS04] Manindra Agrawal, Neeraj Kayal, and Nitin Saxena. Primes is in p. *Annals of Mathematics*, 160(2):781–793, 2004. {4}

[LP19] Hendrik W. Lenstra and Carl Pomerance. Primality testing with gaussian periods. *Journal of the European Mathematical Society*, 21(4):1229–1269, 2019. Previously cited as a 2005 preprint; establishes the $\tilde{O}(\log^6 n)$ deterministic bound. {4}

[NC10] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary edition edition, 2010. {3, 5, 43}

[SS77] Robert Solovay and Volker Strassen. A fast monte-carlo test for primality. *SIAM Journal on Computing*, 6(1):84–85, 1977. See also Erratum: SIAM J. Comput., 7(1), 118 (1978). {4}